



National University of Computer and Emerging Sciences, Lahore



VOX

Maryam Salman 23L-0920 BS(CS)

Muhammad Ezaan Ullah 23L-0676 BS(CS)

Supervisor: Mr. Hamad Ul Qudous

Final Year Project

October 20, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: VOX

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: October 10, 2026

Name: Maryam Salman

Signature: 

Name: Muhammad Ezaan Ullah

Signature: 

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

General-purpose audio models learn from large collections of general audio, yet little is known about what happens when they are adapted to a specific domain with a small amount of data. This project studies that effect for music. A pre-trained general audio model is further pretrained on a small set of openly licensed music tracks using a single GPU, and several training configurations are added to observe the effect. The further pretrained model is then tested on music tasks, to measure the effect of further pretraining on music. Public music-specific models serve only as reference points, not as competitors. The aim is to understand how small-scale domain adaptation changes a general audio model, and the findings will be reported once the experiments are complete.

Executive Summary

General-purpose audio models, trained on large collections of general audio, already perform well on music tasks, while music-specific foundation models are trained from scratch on hundreds to hundreds of thousands of hours of music, often private, with multi-GPU budgets. Much less is known about what happens when a general model is adapted to music with only a small, open dataset and modest hardware. This project studies that effect. It does not try to outperform music-specific models; these appear only as reference points that put the size of any change in context. The central question is whether, and how, a small, single-GPU further pre-training step changes a general-audio model on music tasks. Three questions follow from it: whether further pre-training on a small music subset improves performance on music tasks, how the result compares with music-specific models, and how the effect depends on the amount of data.

The project builds on the Masked Modeling Duo (M2D) model, pre-trained on AudioSet. A seeded, artist-capped pipeline selects 10,000 tracks (about 83 hours) from the training split of the Free Music Archive (FMA), with smaller nested subsets of 1,000, 2,500 and 5,000 tracks for studying the effect of data size. Each track is audited and converted with the same front-end that was used to pre-train M2D, so that any change in behavior comes from the training and not from a mismatch in the input. The proposed framework, called M2D-X-Music, further pre-trains M2D on music mixed with background noise from FSD50K. A frozen copy of the original model acts as a regularizing teacher on the clean input to limit overfitting to the small dataset, and a constant-Q transform (CQT) teacher, borrowed from MERT, encourages the model to capture pitch information. All training runs on a single 16 GB GPU.

The study is carried out as a single experiment. The full M2D-X-Music objective is trained on each of the nested subsets with the same training budget, so that the effect of the amount of music can be seen. The further pre-trained checkpoints, the original M2D and the reference models are evaluated with frozen features and a simple linear classifier, using the evaluation toolkit of the M2D authors. The tasks cover genre, instrument, pitch and tagging. The results are summarized as a music score and a gap-closure ratio to the reference models. The linear classifiers are trained with several random seeds, differences are reported with bootstrap confidence intervals, and negative or inconclusive outcomes will be reported openly.

A small web dashboard accompanies the study. It presents the results and lets a visitor run four music tasks on an uploaded or sample clip, comparing the original and the further pre-trained M2D side by side. It reuses the evaluation code and probes of the study, so it shows only what was actually evaluated. The project has clear limits: no model is trained from scratch or fine-tuned, frame-level tasks such as

key detection and beat tracking are excluded, some test sets are small, and the reference models differ in size, data and context length, so comparisons with them are indicative rather than controlled.

No reviewed work applies this recipe to music on a small open subset with a single GPU. The study therefore offers a controlled analysis of what small-scale domain adaptation does to a general-audio model. The track list, seeds and configuration files will be released so that others can repeat and extend the work, which supports SDG 9 (Industry, Innovation and Infrastructure) and SDG 4 (Quality Education) by lowering the compute and data barrier to audio research.

Table of Contents

List of Figures **xi**

List of Tables **xii**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 2

1.3 Definitions, Acronyms, and Abbreviations 3

1.4 Conclusion 4

2 Project Vision **6**

2.1 Problem Domain Overview 6

2.2 Problem Statement 6

2.3 Problem Elaboration 7

2.4 Goals and Objectives 7

2.5 Project Scope 8

2.6 Sustainable Development Goal (SDG) 10

2.6.1 SDG 9: Industry, Innovation and Infrastructure 10

2.6.2 SDG 4: Quality Education 10

2.7 Constraints 11

2.8 Business Opportunity 11

2.9 Stakeholders Description/ User Characteristics 12

2.9.1 Stakeholders Summary 12

2.9.2 Key High-Level Goals and Problems of Stakeholders 12

2.10 Conclusion 13

3 Literature Review / Related Work **14**

3.1 Definitions, Acronyms, and Abbreviations 14

3.2 Detailed Literature Review 15

3.2.1 Masked Autoencoders that Listen . . . 15
3.2.2 BEATs: Audio Pre-Training with Acoustic Tokenizers . . . 16
3.2.3 Masked Modeling Duo: Learning Representations by Encouraging Both Networks to Model the Input . . . 17
3.2.4 Masked Modeling Duo: Towards a Universal Audio Pre-training Framework (M2D-X) . . . 18
3.2.5 Investigating Design Choices in Joint-Embedding Predictive Architectures for General Audio Representation Learning . . . 19
3.2.6 Audio-JEPA: Joint-Embedding Predictive Architecture for Audio Representation Learning . . . 20
3.2.7 Masked Latent Prediction and Classification for Self-Supervised Audio Representation Learning (MATPAC) . . . 20
3.2.8 MATPAC++: Enhanced Masked Latent Prediction for Self-Supervised Audio Representation Learning . . . 21
3.2.9 Codified Audio Language Modeling Learns Useful Representations for Music Information Retrieval (JukeMIR) . . . 22
3.2.10 Supervised and Unsupervised Learning of Audio Representations for Music Understanding (MULE) . . . 23
3.2.11 MERT: Acoustic Music Understanding Model with Large-Scale Self-Supervised Training . . . 24
3.2.12 A Foundation Model for Music Informatics (MusicFM) . . . 25
3.2.13 MuQ: Self-Supervised Music Representation Learning with Mel Residual Vector Quantization . . . 26
3.2.14 PupuJEPA / PupuM2D: Frequency-Aware Self-Supervised Music Representation Learning . . . 26
3.2.15 Robust wav2vec 2.0: Analyzing Domain Shift in Self-Supervised Pre-Training . 28
3.2.16 Self-Distillation for Further Pre-training of Transformers . . . 29
3.2.17 SONAR: Self-Distilled Continual Pre-training for Domain Adaptive Audio Representation . . . 30
3.2.18 Audio Set: An Ontology and Human-Labeled Dataset for Audio Events . . . 30
3.2.19 FMA: A Dataset for Music Analysis . . . 31
3.2.20 FSD50K: An Open Dataset of Human-Labeled Sound Events . . . 32
3.2.21 Music4All: A New Music Database and Its Applications . . . 33
3.2.22 MARBLE: Music Audio Representation Benchmark for Universal Evaluation . . 33

3.2.23 HEAR: Holistic Evaluation of Audio Representations . . . 35
3.3 Literature Review Summary Table . . . 36
3.4 Conclusion . . . 37

4 Software Requirement Specifications 39
4.1 List of Features . . . 39
4.2 Functional Requirements . . . 40
4.3 Quality Attributes . . . 41
4.3.1 Usability . . . 41
4.3.2 Correctness and Consistency . . . 41
4.3.3 Reliability . . . 41
4.3.4 Maintainability and Extensibility . . . 42
4.3.5 Portability . . . 42
4.4 Non-Functional Requirements . . . 42
4.4.1 Performance . . . 42
4.4.2 Capacity and Concurrency . . . 42
4.4.3 Availability . . . 42
4.4.4 Security and Privacy . . . 43
4.4.5 Reusability . . . 43
4.4.6 Compatibility . . . 43
4.5 Assumptions . . . 43
4.6 Use Cases . . . 44
4.7 Hardware and Software Requirements . . . 48
4.7.1 Hardware Requirements . . . 48
4.7.2 Software Requirements . . . 48
4.8 Graphical User Interface . . . 49
4.9 Database Design . . . 53
4.9.1 ER Diagram . . . 53
4.9.2 Data Dictionary . . . 54
4.10 Risk Analysis . . . 55
4.10.1 Technical Risks . . . 55
4.10.2 Schedule Risks . . . 56
4.10.3 Legal and Licensing Risks . . . 56
4.10.4 Misinterpretation Risks . . . 56
4.11 Conclusion . . . 57

5 Proposed Approach and Methodology **58**

5.1 Dataset Acquisition . . . 58

5.1.1 Choice of Corpus . . . 58

5.1.2 Archives and Metadata . . . 59

5.1.3 Eligibility Rules and Subset Selection . . . 59

5.1.4 Background-Noise Corpus . . . 60

5.2 Data Preprocessing Pipeline . . . 60

5.2.1 Audit and Decoding . . . 61

5.2.2 Front-End . . . 61

5.2.3 Cropping and Normalization . . . 62

5.2.4 Musical-Teacher Targets . . . 63

5.2.5 Storage, Manifest and Integrity Checks . . . 63

5.3 Further Pre-training Framework . . . 63

5.3.1 Initialization . . . 64

5.3.2 Masked Latent Prediction on Noisy Music . . . 64

5.3.3 Background Noise and Denoising . . . 65

5.3.4 Frozen-Teacher Regularization . . . 65

5.3.5 Musical Teacher . . . 66

5.3.6 Joint Objective and Optimization . . . 66

5.4 Experimental Design . . . 68

5.5 Evaluation Protocol . . . 68

5.6 Conclusion . . . 69

6 High-Level and Low-Level Design **71**

6.1 System Overview . . . 71

6.1.1 Data Preparation Pipeline . . . 71

6.1.2 Further Pre-training Framework . . . 71

6.1.3 Evaluation and Analysis . . . 72

6.1.4 Demonstration Dashboard . . . 72

6.2 Design Considerations . . . 72

6.2.1 Assumptions and Dependencies . . . 72

6.2.2 General Constraints . . . 73

6.2.3 Goals and Guidelines . . . 73

6.2.4 Development Methods . . . 74

6.3 System Architecture . . . 74

6.3.1 Subsystem Architecture 74
6.3.2 Sequence Diagrams 76
6.4 Architectural Strategies 77
6.4.1 Extending the M2D Code Base 77
6.4.2 Offline Precomputation of Inputs and Targets 78
6.4.3 Experiments as Command Lines and Run Folders 79
6.4.4 One Evaluation Interface for All Models 79
6.4.5 Files as Interfaces and Single-Worker Serving 79
6.5 Domain Model/Class Diagram 80
6.6 Policies and Tactics 81
6.6.1 Choice of Tools 81
6.6.2 Coding Guidelines and Conventions 81
6.6.3 Testing 81
6.6.4 Requirements Traceability 82
6.6.5 Source Organization and Build 82
6.6.6 Maintenance 82
6.7 Conclusion 82

List of Figures

2.1 **Sustainable Development Goal 9: Industry, Innovation and Infrastructure** 10
2.2 **Sustainable Development Goal 4: Quality Education** 10

4.1 **Use-case diagram of the dashboard** 44
4.2 **Navigation flow of the dashboard** 50
4.3 **Wireframe of the Results Overview** 50
4.4 **Wireframe of the Demo screen: task and input** 51
4.5 **Wireframe of the Demo screen: output** 51
4.6 **Wireframe of the comparison view** 52
4.7 **Wireframe of the About page** 52
4.8 **ER diagram of the dashboard database** 53

5.1 **Overview of the proposed methodology** 58
5.2 **Data acquisition and preprocessing pipeline** 61
5.3 **Architecture of the further pre-training framework** 64

6.1 **Layered system architecture** 75
6.2 **Modules of the further pre-training subsystem** 76
6.3 **Sequence diagram of one further pre-training step** 77
6.4 **Sequence diagram of a dashboard prediction request** 78
6.5 **Class diagram** 80

List of Tables

3.1 Summary of the reviewed literature 36

4.1 Functional requirements of the dashboard 40
4.2 UC-01 View Research Results 45
4.3 UC-02 Select a Downstream Task 45
4.4 UC-03 Provide Audio Input 46
4.5 UC-04 Run Inference and View Output 46
4.6 UC-05 Compare Two Models on a Clip 47
4.7 UC-06 Import Evaluation Results 47
4.8 Data dictionary of the dashboard database 54
4.9 Summary of risks of the dashboard 56

5.1 FMA audio archives and their use in this study 59
5.2 Front-end and target parameters 62
5.3 Fixed settings of the further pre-training runs 67
5.4 Pre-training arms 68
5.5 Downstream tasks 69

Chapter 1 Introduction

Self-supervised learning (SSL) lets a model learn useful representations from unlabeled data by solving a task built from the data itself, for example predicting masked parts of the input [1, 2], which removes the need for costly manual annotation. In audio, SSL has produced foundation models such as Masked Modeling Duo (M2D) [3, 4], which is pre-trained once on a large general-audio dataset, AudioSet [5], and then reused for many downstream tasks in environmental sound, speech and music, often with the encoder kept frozen and only a small classifier trained on top [6]. Reusing one pre-trained encoder is cheaper and more practical than training a separate model for each task, which is usually held back by scarce labeled data and high training costs.

Music-specific foundation models, such as MERT [7], MusicFM [8], MuQ [9] and PupuM2D [10], perform strongly on music tasks. They are, however, pre-trained from scratch on large music collections, for example about 20,000 hours for the public MERT-v1-95M model [11] and 100,000 to 160,000 hours of mostly private in-house music for the largest models [7, 8, 10], with multi-GPU budgets that few research groups have. At the same time, recent comparisons show that general-audio models trained by masked latent prediction are already close to these music models, and on some tasks above them [12, 10].

This project explores a cheaper middle path. Instead of training a music model from scratch, it further pre-trains the general-audio M2D model on a small, open music dataset, a 10,000-track subset of the Free Music Archive (FMA) [13], using a single GPU. Further pre-training on a small domain dataset risks overfitting and the loss of general ability [14], and the M2D authors proposed the M2D-X recipe, which adds background noise and a frozen copy of the original model as a regularizer, for exactly this setting [4]. The project extends this recipe with a musical teacher in the style of MERT and studies how much it helps on music tasks and which problems arise along the way. A small web dashboard demonstrates the results and four music tasks. This chapter states the purpose of the report, its intended audience, the terms used, and the structure of the remaining chapters.

1.1 Purpose of this Document

The purpose of this document is to present the motivation, requirements, methodology and design of our Final Year Project (FYP), which investigates further pre-training of the AudioSet-pretrained M2D model on music; the implementation, test cases and experimental results are reported in the later chapters as the work proceeds. The project develops M2D-X-Music, a framework that combines the background-noise and frozen-teacher regularization recipe of M2D-X [4] with a constant-Q transform (CQT) musical teacher in the style of MERT [7]. The framework is trained on 10,000 tracks (about 83 hours) of

FMA [13], with background noise from FSD50K [15], on a single 16 GB GPU, and the resulting models are compared with the original M2D and with public music-specific models.

The central research question is: Can a small, open, single-GPU further pre-training step bring a general-purpose audio model closer to music-specific foundation models on music tasks? It is broken down into four sub-questions:

- Does further pre-training a general-audio foundation model on a small music dataset improve its frozen-feature performance on music tasks, and how much of the gap to music-specific models does it close?
- Which components of M2D-X, namely background noise and the frozen-teacher regularizer, are needed on music data?
- Does a CQT musical teacher add value beyond M2D-X, especially on pitch-related tasks?
- How does the benefit depend on the number of FMA tracks?

The methods are a seeded data pipeline, a set of compute-matched further pre-training runs that differ in one factor at a time, and a frozen-feature linear-probe evaluation on music tasks. The main limitations are the small test sets of some tasks, the uncontrolled differences between the reference models, and the single-GPU budget; the main assumption is that the public M2D checkpoint and the FMA audio are available and licensed for research use. Section 1.4 outlines the order in which the report presents this work.

1.2 Intended Audience

The report is written for the following readers:

- FYP evaluation panel and supervisor, who judge the project for technical soundness, reproducibility and clarity.
- Researchers in audio representation learning and music information retrieval (MIR), who may reproduce the experiments or extend them with other teachers, base models or larger subsets.
- Researchers in other data-scarce audio domains, such as bioacoustics, medical sounds (for example respiratory or heart sounds), low-resource speech, and regional or traditional music, who may reuse the recipe to adapt a general-audio model to their own small datasets.
- Developers and companies in music technology, such as streaming, recommendation and tagging services, startups and independent developers, who need good music embeddings but cannot afford to pre-train a music foundation model from scratch.

- Music archives, libraries and broadcasters, who need to organize and tag large audio collections at low cost.
- Universities and research groups with limited computing resources, for whom a single-GPU recipe on open data is a realistic starting point.
- Students and educators, including future FYP groups who may continue this work and teachers who need a reproducible example of modern self-supervised audio learning.
- Visitors of the demonstration dashboard, including non-specialists, who can explore the results and try the music tasks without reading the technical chapters.

Readers of the technical chapters are assumed to know the basics of deep learning, such as transformers, self-supervised learning and linear probing, but not the details of M2D, MERT or FMA, which the report introduces where needed.

1.3 Definitions, Acronyms, and Abbreviations

AMP: Automatic Mixed Precision, training with 16-bit and 32-bit numbers to save memory

API: Application Programming Interface

CPU: Central Processing Unit

CQT: Constant-Q Transform, a spectral transform with logarithmically spaced frequency bins, suited to pitch

CSV: Comma-Separated Values, a plain-text table format

DINO: Self-distillation with no labels, a self-supervised method whose classification head is used in MATPAC

EMA: Exponential Moving Average

ER diagram: Entity-Relationship diagram of a database

ESC-50: Environmental Sound Classification dataset with 50 classes

EVAR: Evaluation of Audio Representations, the evaluation toolkit of the M2D authors

FMA: Free Music Archive, an open music dataset

FR: Functional Requirement

FSD50K: Freesound Dataset 50K, used as the background-noise corpus

FYP: Final Year Project

GCR: Gap-Closure Ratio

GPU: Graphics Processing Unit

GTZAN: A music genre dataset of 1,000 clips in 10 genres

HEAR: Holistic Evaluation of Audio Representations, an audio benchmark

HTML: HyperText Markup Language

JEPA: Joint-Embedding Predictive Architecture

JSON: JavaScript Object Notation, a text format for exchanging data

kNN: k-Nearest-Neighbor classifier

Linear probe: A single linear classifier trained on frozen encoder features

M2D: Masked Modeling Duo, a masked-latent-prediction audio model

M2D-X: The extension of M2D for specialized applications, with background noise and an offline network

M2D-X-Music: The further pre-training framework proposed in this report

MAE: Masked Autoencoder

mAP: Mean Average Precision

MARBLE: Music Audio Representation Benchmark for Universal Evaluation

MATPAC / MATPAC++: Masked latent Prediction And Classification, and its enhanced version

MERT: Music undERstanding model with large-scale self-supervised Training

MIR: Music Information Retrieval

MLP: Multi-Layer Perceptron

MTT: MagnaTagATune, a music tagging dataset

NSynth: A dataset of 305,979 single musical notes with instrument and pitch labels

ORM: Object-Relational Mapping

RQ: Research Question

RVQ: Residual Vector Quantization

SDG: Sustainable Development Goal

SQLite: A file-based relational database engine

SSL: Self-Supervised Learning

STFT: Short-Time Fourier Transform

UC: Use Case

ViT: Vision Transformer

1.4 Conclusion

This chapter introduced the problem and the motivation of the project. Music-specific foundation models perform strongly but rely on large, often private, music corpora and multi-GPU training, whereas general-audio models such as M2D are public and already strong on music. The project therefore investigates a low-cost middle path: further pre-training the AudioSet-pretrained M2D on 10,000 open FMA

tracks on a single GPU with M2D-X-Music, which combines background noise and a frozen-teacher regularizer with a CQT musical teacher. Four research questions guide the study: the music gain and the remaining gap to music-specific models, the role of each M2D-X component, the value of the CQT teacher, and the effect of the amount of data.

The rest of the report is organized as follows. Chapter 2 presents the project vision: the problem statement, goals and objectives, scope, sustainable development goals, constraints and stakeholders. Chapter 3 reviews the related research, datasets and benchmarks, and identifies the gap that this project addresses. Chapter 4 specifies the requirements of the demonstration dashboard, including its use cases, interface wireframes, database design and risks. Chapter 5 describes the proposed approach: the data pipeline, the M2D-X-Music framework, the experimental design and the evaluation protocol. Chapter 6 presents the high-level and low-level design of the system. Chapters 7 to 10 (implementation and test cases, user manual, experimental results and discussion, and conclusions) cover work that is carried out later in the project.

Chapter 2 Project Vision

To make high-quality music audio representations accessible to everyone, by showing that a general-purpose audio model can be adapted to music with a small, open dataset and a single GPU, without the large private corpora and multi-GPU budgets that current music foundation models require.

This chapter sets out what the project is trying to achieve and why. It describes the problem domain, states and elaborates the problem, lists the goals and objectives, and defines the scope. It then relates the project to the United Nations Sustainable Development Goals and describes the constraints, the opportunity that the project addresses, and its stakeholders.

2.1 Problem Domain Overview

The project belongs to music information retrieval (MIR) and audio representation learning. An audio encoder converts a recording into feature vectors that downstream tasks, such as genre classification, instrument recognition or tagging, can use with a very simple classifier [6, 16]. The system built in this project consists of an experimental research pipeline and a small demonstration dashboard. The pipeline acquires a 10,000-track subset of the Free Music Archive [13], audits it, and converts it into log-mel spectrograms and CQT targets. It then further pre-trains the publicly available M2D model [3], which was trained on AudioSet [5], with the M2D-X-Music objective: music mixed with background noise from FSD50K [15] as input, a frozen copy of the original model as a regularizing teacher on the clean input [4], and a CQT regression head as a musical teacher [7]. Finally, it evaluates the resulting checkpoints and public music-specific reference models, namely MERT [7], MusicFM [8] and MAT-PAC++ [12], with frozen features and linear probes on genre, instrument, pitch and tagging tasks. All training and evaluation runs on one 16 GB GPU. The dashboard presents the results and lets a visitor try four music tasks on an audio clip (Chapter 4).

2.2 Problem Statement

Music-specific foundation models achieve strong results but need very large, often private, music corpora and multi-GPU training [7, 8, 10]. This makes them expensive to develop and impossible for most of the research community to reproduce, because the training data are not publicly available. General-audio models such as M2D are public and already strong on music [4, 12], yet it is unknown whether a small, cheap, music-specific further pre-training step can improve them on music tasks, given that naive continued pre-training on a small dataset can overfit and degrade a model [14].

2.3 Problem Elaboration

The problem splits into five sub-problems, which correspond to the research questions of Section 1.1.

1. A legal, full-quality, artist-filtered music corpus must be selected and converted, with at most five tracks per artist, quality audits and nested subsets, so that every experiment draws on identical data. The audio must reach the model through exactly the same front-end as in the original pre-training [17], or a distribution shift would be confused with the effect under study.

2. Further pre-training on about 10,000 clips (about 83 h, or 1.5 % of the 5,569 h of AudioSet used for M2D) risks overfitting, and earlier work shows that naive continuation can degrade a model [4, 14]. The project must test which of background noise and the frozen-teacher regularizer are needed on music (RQ1, RQ2).

3. MERT showed that a CQT teacher improves pitch-related tasks such as key detection [7]. It is unknown whether the same teacher helps inside the M2D framework on a small dataset (RQ3).

4. The reference models differ in size, data, sampling rate and context length, and some test sets are small; for example, the fault-filtered GTZAN test split has only 290 clips [18]. The evaluation must use one frozen-feature protocol and report uncertainty.

5. The project must show how much of the benefit depends on the number of tracks, from 1,000 to 10,000 (RQ4).

No reviewed work combines background noise, a frozen-teacher regularizer and a CQT teacher on a small open subset with one GPU (Chapter 3); this is the gap that the project addresses.

2.4 Goals and Objectives

The goals of Vox are as follows:

- Show whether, and to what extent, a single-GPU further pre-training step on a small open music subset can improve a general-purpose masked-latent-prediction audio model on music tasks and bring it closer to music-specific foundation models.
- Understand which parts of the further pre-training recipe are needed to avoid overfitting on a small music dataset.
- Make the findings easy to reproduce and to explore, through open data, released configuration files and a demonstration dashboard.

The objectives of Vox are as follows:

1. Build a reproducible, resumable pipeline that selects, audits and converts 10,000 FMA tracks (nested subsets of 1,000, 2,500, 5,000 and 10,000) and a background-noise corpus into M2D-compatible log-mel arrays and CQT targets.
2. Implement the M2D-X-Music objective on top of the official M2D code, adding the CQT musical teacher.
3. Train a set of compute-matched further pre-training runs that add the components one at a time (naive continuation, background noise, the frozen-teacher regularizer, the CQT teacher, and the CQT teacher without the regularizer), together with runs on 1,000, 2,500 and 5,000 tracks, each processing 6.0 million sample views.
4. Evaluate all checkpoints and reference models on GTZAN, FMA-small, NSynth (instrument and pitch), and MagnaTagATune with a six-seed linear probe, bootstrap intervals and a pre-declared rule for calling a difference supported.
5. Report the music gain and the gap-closure ratio to MERT, MATPAC++ and MusicFM, and state the limits of every comparison.
6. Develop a demonstration dashboard that presents the results and runs four MARBLE music tasks (genre, tagging, instrument and pitch) on an audio clip.
7. Release the track list, seeds and configuration files so that others can repeat the study.

2.5 Project Scope

The scope of this project is to design, implement and evaluate a further pre-training study of M2D on music, and to build a small dashboard that demonstrates its results. The project focuses on adapting the AudioSet M2D checkpoint to music data and on evaluating the resulting models with frozen-feature linear probes. It involves the following activities:

- Download the FMA metadata and medium audio archive and the FSD50K development set, verify their checksums, and select 10,000 tracks from the training split with a seeded, artist-capped procedure.
- Convert the audio into log-mel spectrograms and CQT targets, and run integrity checks to ensure data quality and consistency.
- Further pre-train the AudioSet M2D checkpoint with the specified configurations on a single NVIDIA RTX A4000 GPU with 16 GB of memory.
- Evaluate the resulting checkpoints and publicly available reference models with frozen-feature

linear probes.

- Analyze the results with respect to research questions RQ1 to RQ4, including negative or inconclusive outcomes.
- Build a demonstration dashboard that shows the results and runs four music tasks with the trained models.

The project is developed in Python with PyTorch [19], the official M2D and EVAR code bases [17, 20] and nnAudio [21]; the dashboard uses FastAPI and SQLite. Development is organized in short, iterative milestones, and a pilot run of a few epochs validates the training pipeline and confirms that the experiments fit the computational budget before the main runs start.

The project deliverables are:

- The data-processing pipeline code, the selected track list, random seeds, dataset manifest and configuration files.
- The M2D-X-Music training code and the further pre-trained model checkpoints.
- The evaluation code, including the additional FMA-small and MagnaTagATune task definitions.
- The demonstration dashboard.
- This report, documenting the methodology, design, experimental results, analysis and limitations.

The project scope does not include:

- Pre-training any model from scratch, or developing and training an acoustic tokenizer.
- Fine-tuning any model or reference checkpoint, as all evaluations use frozen features.
- Adapters, low-rank adaptation, predictors with multiple hypotheses and classification heads; these are left for future work.
- Redistributing FMA audio or derived spectrograms.
- A production or commercial user-facing application; the dashboard is a local demonstration only.
- Tasks beyond the five evaluation tasks, such as key detection, beat tracking or emotion recognition.

This scope serves as a guide throughout the implementation and evaluation, so that the work stays focused on the research questions, the available computational resources and the defined deliverables.

2.6 Sustainable Development Goal (SDG)

The United Nations Sustainable Development Goals are seventeen global goals adopted in the 2030 Agenda for Sustainable Development [22]. The primary SDG addressed by this project is:

2.6.1 SDG 9: Industry, Innovation and Infrastructure

The project mainly targets SDG 9, shown in Figure 2.1, in particular target 9.5 on enhancing scientific research and upgrading technological capabilities [22]. By testing whether a strong music representation model can be obtained from public data and one GPU, the project lowers the compute and data barrier to audio research and supports researchers and developers outside large, well-funded laboratories.



Figure 2.1: Sustainable Development Goal 9: Industry, Innovation and Infrastructure.

Additionally, the project also supports:

2.6.2 SDG 4: Quality Education

The project also supports SDG 4, shown in Figure 2.2, because the open code, track list and configuration files, together with the dashboard, give students and teachers a reproducible, hands-on example of modern self-supervised audio learning.



Figure 2.2: Sustainable Development Goal 4: Quality Education.

2.7 Constraints

The project is subject to the following constraints:

- One NVIDIA RTX A4000 with 16 GB of memory. The micro-batch may have to be halved with doubled gradient accumulation, and the number of epochs reduced for all runs alike if the throughput is low.
- The FYP calendar limits the number of further pre-training runs to the core comparisons of the framework components and the data-size runs. Optional runs are added only if time remains, and the order of the runs is fixed in advance so that the most informative ones finish first.
- Only 10,000 FMA tracks (about 83 hours) are used, from the training split of the medium subset; FMA genre labels are noisy, and experimental music is under-represented [13].
- FMA audio is released under Creative Commons licenses and kept local; the MERT weights are licensed for non-commercial use only and are used for comparison only [11].
- The M2D code has no CQT teacher, and EVAR has no FMA-small or MagnaTagATune tasks, so both must be added; the CQT settings must match the MERT implementation [23].
- The reference models differ in size, data and context length, and some test sets are small, so comparisons with the references are indicative, not controlled.

2.8 Business Opportunity

Although this is a research project, its result has practical value. Music platforms, startups and independent developers need audio embeddings for tagging, genre and instrument classification, recommendation and search, but cannot afford to pre-train music foundation models from scratch. If a further pre-training step on about 83 hours of music recovers a useful part of the gap to music-specific models, a team can adapt a public general-audio checkpoint to its own licensed or in-house catalogue with roughly a day of single-GPU training, a cost that the pilot run of this project will confirm. The same recipe could be reused for other narrow domains with scarce data, such as podcasts, regional music traditions or bioacoustics. The released pipeline and track list can also serve as a low-cost baseline for later work, and the dashboard shows the idea to non-specialists.

2.9 Stakeholders Description/ User Characteristics

The system is a research pipeline with a demonstration dashboard. Its direct users are the people who run, evaluate and build on it, and its indirect beneficiaries are the researchers and developers who can reuse the recipe and the findings.

2.9.1 Stakeholders Summary

The project team, the two students who carry out the project, build the pipeline, run training and evaluation, develop the dashboard and write the report. The supervisor, the faculty member who supervises the project, guides the research design and checks its scientific soundness, while the FYP evaluation panel, the faculty members who assess the project, judge the rigor, reproducibility and clarity of the work. Beyond the project itself, audio and MIR researchers, who study audio and music representations, may reuse the code, checkpoints and findings, or compare their own models against them. Researchers in other data-scarce audio domains, such as bioacoustics, medical audio and low-resource speech, may apply the recipe to adapt a general-audio model to their own small datasets, and music-application developers, the companies, startups and independent developers who build music services, may adopt it to adapt general-audio models to their own catalogues. Dashboard visitors, the evaluators, students and non-specialists who use the dashboard, explore the results and try the four music tasks. Finally, the dataset and model providers, namely the FMA artists and maintainers, the FSD50K maintainers, and the authors of M2D and MERT, supply the data and models, and their licenses must be respected.

2.9.2 Key High-Level Goals and Problems of Stakeholders

The main goal of each stakeholder, and the problem that stands in its way, are described below.

- The project team aims to finish a valid, reproducible study within the single-GPU budget. Its problem is limited compute, time and data, together with the risk of overfitting.
- The supervisor wants clear hypotheses that the data can contradict. The problem is avoiding conclusions that rest on small test sets or a single seed.
- The FYP evaluation panel expects a verifiable method, honest results and stated limits. The problem is that uncontrolled references and inconclusive differences must be flagged.
- Audio and MIR researchers aim to reproduce the work and extend it with other teachers or models. To do so they need the track list, seeds, configuration files and evaluation protocol.
- Researchers in other data-scarce audio domains want a cheap way to specialize a general-audio model to their data. They have little data and compute, and need to know which parts of the recipe

matter.

- Music-application developers want a cheap way to improve music embeddings. They cannot afford training from scratch and need to know how large the gain is.
- Dashboard visitors want to understand the results quickly and see the models at work. The problem is that the technical chapters are hard to read, and single predictions can mislead.
- Dataset and model providers expect correct use of their licensed material. The problem is that non-commercial and redistribution limits must be respected.

2.10 Conclusion

This chapter defined the vision of the project: a single-GPU study of whether further pre-training M2D on 10,000 FMA tracks, with background noise, a frozen-teacher regularizer and a CQT musical teacher, improves a general-audio foundation model on music tasks. It broke the problem into sub-problems, fixed the goals, objectives, deliverables and exclusions, related the work to SDG 9 and SDG 4, listed the constraints, outlined the business opportunity and described the stakeholders. The next chapter reviews the related research, datasets and benchmarks on which these choices rest.

Chapter 3 Literature Review / Related Work

This chapter reviews the work on which the project builds. It covers general-purpose audio models trained by masked prediction, including the Masked Modeling Duo (M2D) family; music-specific self-supervised models; studies of further pre-training and forgetting; and the datasets and benchmarks that make a study of this kind possible. Each research item is summarized, analyzed critically and related to the proposed work. The chapter ends with a summary table and the research gap that this project addresses.

3.1 Definitions, Acronyms, and Abbreviations

CQT: Constant-Q transform, a spectral transform with logarithmically spaced frequency bins, so that every octave has the same number of bins.

EMA: Exponential moving average, used to update the target network from the online network.

Further pre-training: Resuming self-supervised pre-training of an already pre-trained model on a new, usually smaller, dataset.

Forgetting: The loss of previously learned ability when a model is trained further on new data.

JEPA: Joint-embedding predictive architecture, which predicts the latent representation of hidden regions from a visible context.

Linear probing: Evaluating a frozen encoder by training only a linear layer on its features.

Log-mel spectrogram: A time-frequency representation with mel-spaced frequency bands and logarithmic magnitude.

mAP: Mean average precision, the usual metric for multi-label tagging.

Masked latent prediction: Predicting the latent representations of masked input patches instead of their raw values.

Masking ratio: The fraction of input patches hidden from the online network.

Online and target networks: The network being trained and its slowly updated copy, whose outputs are the prediction targets.

Offline network: In M2D-X, an additional network that sees the clean input and provides an extra training signal.

RVQ: Residual vector quantization, which turns audio features into discrete codes.

SSL: Self-supervised learning, learning from unlabeled data through a pretext task.

ViT: Vision transformer, a transformer encoder applied to patch tokens of an image or spectrogram.

3.2 Detailed Literature Review

The review covers seventeen research studies, four datasets and two benchmarks. They were selected because they answer at least one of four questions: how masked-latent-prediction models such as M2D learn general audio representations and how well these transfer to music; what music-specific models do differently and what they gain; what is known about further pre-training a model on a small domain dataset, including forgetting; and which datasets and protocols allow such a study to be run and compared. The items are ordered by theme: general-audio models (Sections 3.2.1 to 3.2.8), music-specific models (Sections 3.2.9 to 3.2.14), further pre-training (Sections 3.2.15 to 3.2.17), datasets (Sections 3.2.18 to 3.2.21) and benchmarks (Sections 3.2.22 and 3.2.23). Preprints that have not been peer reviewed are marked as such.

3.2.1 Masked Autoencoders that Listen

3.2.1.1 Summary of the research item

Huang et al. [24] extended masked autoencoders to audio spectrograms. About 80 % of the patches are masked, a ViT-Base encoder processes only the visible patches, and a decoder with local-window attention reconstructs the whole spectrogram. After pre-training on AudioSet-2M and fine-tuning the encoder, the model reached an AudioSet-2M mAP of 47.3, 94.1 % accuracy on ESC-50 and 98.3 % on Speech Commands V2.

3.2.1.2 Critical analysis of the research item

Audio-MAE showed that a simple masked-autoencoding design, carried over from images with little change, scales well to audio and reached state-of-the-art fine-tuning results without any external supervised pre-training. Its decoder is also well matched to the input: local-window attention exploits the strong correlation between neighboring time-frequency bins, which purely global attention would not take into account. However, the prediction target is the spectrogram itself, so a large part of the learning signal is spent on low-level acoustic detail rather than on the abstract content of the sound. BEATs was proposed precisely because reconstruction loss is a weak driver of semantic abstraction [25], and the later latent-prediction methods reviewed below share this view. A second limitation concerns the evaluation. The headline results are obtained by fine-tuning the whole encoder on each target dataset, which measures how well the network can be adapted rather than how good its frozen features are, and therefore says little about the representation itself.

3.2.1.3 Relationship to the proposed research work

Audio-MAE is the reconstruction baseline that M2D replaces by latent prediction, and the masked spectrogram model MSM-MAE [26] from which M2D grew belongs to the same family. The code base of M2D is derived from the MAE implementation, which makes the M2D checkpoint a direct descendant of this design.

3.2.2 BEATs: Audio Pre-Training with Acoustic Tokenizers

3.2.2.1 Summary of the research item

Chen et al. [25] train an audio encoder to predict discrete labels of masked patches, where the labels come from an acoustic tokenizer. In the first iteration the tokenizer is a random projection; in later iterations a tokenizer is distilled from the previously trained (or fine-tuned) audio model, and the two are improved alternately. About 75 % of the input is masked. The authors report an AudioSet-2M mAP of 48.6 for a single model and 50.6 for an ensemble without external data, and 98.1 % accuracy on ESC-50.

3.2.2.2 Critical analysis of the research item

BEATs replaced continuous reconstruction targets with discrete labels produced by an acoustic tokenizer, and these semantically rich targets gave strong results with fewer parameters and less data than earlier models. The method also follows a clear iterative recipe in which the tokenizer and the encoder improve each other, so that each round of pre-training provides better targets for the next. However, this strength comes at the price of a multi-stage pipeline. A tokenizer must be trained before the encoder, and the strongest variants distill the tokenizer from a model that was fine-tuned on AudioSet labels, which means that they are not purely self-supervised and that their gains cannot be attributed to the pretext task alone. The cost of the recipe is a further concern: tokenizer training and several rounds of pre-training are far beyond a single-GPU budget, so the approach cannot easily be repeated or adapted to a new domain by a small research group.

3.2.2.3 Relationship to the proposed research work

BEATs is the base model that SONAR continues to pre-train on FMA data (Section 3.2.17), and its discrete-target principle is shared by MERT, MusicFM and MuQ. This project keeps the continuous latent targets of M2D and therefore avoids training or distilling a tokenizer.

3.2.3 Masked Modeling Duo: Learning Representations by Encouraging Both Networks to Model the Input

3.2.3.1 Summary of the research item

Niizumi et al. [3] proposed Masked Modeling Duo (M2D). An online network encodes the visible patches and a predictor, given shared mask tokens, estimates the representations of the masked patches. The target network, an EMA of the online encoder, encodes only the masked patches, so that the target carries no information about the visible patches and both networks are pushed to model the input. The loss is the mean squared error between ℓ_2-normalized prediction and standardized target. The released model uses a ViT-Base encoder on 80×608 log-mel inputs (6 s at 16 kHz) with 16×16 patches and a masking ratio of 0.7, pre-trained for 300 epochs on AudioSet (2,005,132 clips, 5,569 h). The conference paper reports state-of-the-art linear-evaluation results on four of nine tasks (UrbanSound8K, VoxCeleb1, CREMA-D and GTZAN); in the journal extension [4], the model with masking ratio 0.7 reached an average of 80.2 over nine linear-evaluation tasks, against 79.1 for the MAE-based predecessor MSM-MAE, with 84.1 % on GTZAN.

3.2.3.2 Critical analysis of the research item

M2D is a simple method supported by ablations of the masking ratio, the target input and the patch size, so the contribution of each design choice is visible. Its central idea, encoding only the masked patches in the target network, prevents shortcuts that would make prediction too easy and pushes both networks to model the input. The ViT-Base encoder, with about 86 M parameters, is also small enough to be further pre-trained on a single GPU. However, pre-training from scratch needs a large dataset and a multi-GPU budget, so the recipe cannot be repeated on a new domain by a small group. The learned features are also abstract: compared with the MAE, the authors observe worse results on pitch classification (Surge) in linear evaluation and on VoxCeleb1 in fine-tuning, and conclude that M2D representations are less sensitive to pitch. Finally, the model is restricted to a fixed 6-s input and to random masking only.

3.2.3.3 Relationship to the proposed research work

M2D is the pre-trained model of this study. The checkpoint M2D/0.7, its front-end, its loss and its random-masking setting are reused unchanged as the starting point and the base objective of the further pre-training (Chapter 5). The reported insensitivity to pitch is one reason for adding a pitch-oriented musical teacher.

3.2.4 Masked Modeling Duo: Towards a Universal Audio Pre-training Framework (M2D-X)

3.2.4.1 Summary of the research item

In the journal extension, Niizumi et al. [4] redefine M2D as a framework for learning representations specialized to an application X, including cases with little data. M2D-X adds background noise to the input of the online and target networks, which acts as augmentation, and an offline network that receives the clean input and provides a configurable additional task: supervised learning of labels, distillation of a domain model, or regularization with the original pre-trained model. Because the offline target comes from the clean signal, the noise turns the task into denoising distillation, an idea first used to specialize M2D to speech [27]. The total loss is the weighted sum $\lambda_{\text{m2d}}\mathscr{L}_{\text{m2d}} + \lambda_{\text{off}}\mathscr{L}_{\text{off}}$. The framework was validated on AudioSet with supervised labels (M2D-AS, average 80.6 in linear evaluation), on speech, and on respiratory sounds. In the last case, the AudioSet model was further pre-trained on 539 ICBHI 2017 recordings, with the training list replicated to 10,000 samples per epoch for 600 epochs: continuing with the M2D loss alone and no noise lowered the score from 61.93 to 61.17, background noise at a ratio of 0.3 raised it to 62.73, and the full M2D-X with the frozen original model as offline teacher reached 63.29.

3.2.4.2 Critical analysis of the research item

M2D-X provides the main published recipe for further pre-training a masked-latent-prediction audio model on a very small dataset, and it is supported by careful ablations that separate the effect of background noise from that of the offline regularizer. The further pre-training is also designed to run on a single GPU, which makes it practical outside large laboratories. However, the evidence for the recipe is narrow. The further pre-training result rests on one small medical dataset, with a similar trend on a second respiratory dataset (SPRSound), and its gains of one to two points are of the size of the variation between runs, so they are suggestive rather than conclusive. No music data were used, and the study does not measure what happens to general-audio ability after further pre-training, which leaves the question of forgetting open. The noise ratio and the loss weights are also chosen per application rather than studied systematically.

3.2.4.3 Relationship to the proposed research work

M2D-X is the methodological foundation of Chapter 5. The proposed framework keeps its noise injection, offline network and frozen-teacher regularization, adds a music-specific teacher, and tests whether the recipe carries over from a respiratory database of 5.5 h to about 83 h of music.

3.2.5 Investigating Design Choices in Joint-Embedding Predictive Architectures for General Audio Representation Learning

3.2.5.1 Summary of the research item

Riou et al. [28] studied JEPA-style pre-training of a ViT-Base on AudioSet spectrograms and evaluated on environmental, speech and music tasks. They compared unstructured, multi-block and time masking, and compared masking the target in the audio domain with masking it only in the latent domain. Unstructured random masking outperformed the alternatives on nearly all tasks, in contrast to images, and masking the target spectrogram before the target encoder was, in most tasks, better than feeding it the whole input. The duration of the training audio mattered: with 2.1 s, 3.2 s and 6.4 s inputs, GTZAN accuracy rose from 82.1 to 84.3 and 86.9 while NSynth accuracy fell from 76.8 to 75.6 and 74.9.

3.2.5.2 Critical analysis of the research item

The main strength of this study is a systematic ablation of the design choices that matter for audio, namely the masking strategy, the masking of the target and the input duration, all evaluated with a common toolkit on eight tasks. Because the task set covers music (GTZAN and NSynth) as well as environmental sound and speech, it reveals that music tasks can react in opposite directions to the same design change: a longer input helped genre classification but lowered NSynth accuracy. This is a useful warning that a single music score can hide such trade-offs. However, the scope of the study is limited. Only AudioSet was used for pre-training and only one backbone size was tested, so it is unclear whether the conclusions hold for music-specific data or objectives. The exploration also covered only a few masking strategies and three input durations, so the space of possible designs is sampled rather than mapped.

3.2.5.3 Relationship to the proposed research work

The findings justify keeping random masking and masked-only target encoding in the framework, and they show that a longer input favors genre-like (global) tasks while shorter inputs favor note-level tasks such as instrument and pitch. The 6.08-s window of M2D therefore suits genre tasks, and its effect on pitch tasks is one of the questions that the NSynth pitch evaluation will probe.

3.2.6 Audio-JEPA: Joint-Embedding Predictive Architecture for Audio Representation Learning

3.2.6.1 Summary of the research item

Tuncay et al. [29] translated the image JEPA to audio with a ViT-Base (16×16 patches) and random patch masking of 40–60 %. They pre-trained on 1,921,982 AudioSet clips at 32 kHz (5,338 h) for about 13 epochs and evaluated with the 21-dataset X-ARES suite [30]. With an MLP probe the model was first or second on some tasks and last on about half, with particular weakness on speech commands; with a k-nearest-neighbor probe it ranked first on ESC-50, FMA-small and GTZAN among three models, ahead of wav2vec 2.0 and data2vec despite using under a fifth of their training data.

3.2.6.2 Critical analysis of the research item

Audio-JEPA shows that a competitive general-audio encoder can be obtained at a very small training cost: about 13 epochs on AudioSet were enough to rank first among three models on ESC-50, FMA-small and GTZAN with a k-nearest-neighbor probe. It is also one of the few studies to include FMA-small and GTZAN in a standardized evaluation suite, which makes its music results easy to place. However, the evidence is limited. The training was short, no hyper-parameters were tuned and only two baselines (wav2vec 2.0 and data2vec) were compared, and the work is an arXiv preprint written for a challenge rather than a peer-reviewed study. The design also has a structural weakness: the target encoder sees the whole spectrogram, and the authors note that the embedding space is not guaranteed to be linearly separable, which hurts linear probing. Its uneven results with an MLP probe, last on about half of the tasks, show that the cheap training does not come for free.

3.2.6.3 Relationship to the proposed research work

The study illustrates the cost of not following the M2D rule of masking the target input and of weak linear separability of JEPA features. It also shows FMA-small used as a standard evaluation task, which this project uses as its in-domain genre task.

3.2.7 Masked Latent Prediction and Classification for Self-Supervised Audio Representation Learning (MATPAC)

3.2.7.1 Summary of the research item

Quelennec et al. [31] combine masked latent prediction with an unsupervised classification task. The predicted and the target latents of the masked patches are each projected by a DINO-style head [32] into probability distributions over 2,048 prototypes that are matched by cross-entropy, with centering and

sharpening to prevent collapse. The total loss is $(1-\alpha)\mathscr{L}_{\text{cls}} + \alpha\mathscr{L}_{\text{pred}}$ with $\alpha = 0.5$, and pre-training follows M2D (ViT-Base, 6-s inputs of AudioSet). In linear probing over seven tasks the average was 74.9, against 74.0 without the classification task and 73.7 for M2D, and the model reached 41.1 mAP on MagnaTagATune, above the supervised and self-supervised methods compared.

3.2.7.2 Critical analysis of the research item

MATPAC improves the latent space of masked latent prediction at no inference cost, since the classification head is used only during pre-training, and the authors support the design with an ablation of the loss weight and of the number of prototypes. Its strong music-tagging result on MagnaTagATune, obtained by a model trained only on general audio, is also notable evidence that general-audio pre-training transfers well to music. However, the authors themselves conclude that the classification task mainly acts as a regularizer rather than as an independent source of information, and without the prediction task the model collapses, so the added component cannot stand on its own. The improvements over M2D are also small, about one point on average, and depend on the version of AudioSet available to the authors, because part of AudioSet is no longer downloadable. Differences of this size may therefore reflect the training data as much as the method, and they should be interpreted with caution.

3.2.7.3 Relationship to the proposed research work

MATPAC is the closest competitor to M2D within masked latent prediction. Its design shows that an extra objective on top of the prediction loss can improve the representation, which parallels the extra offline objectives of this project, but the classification head and prototypes are not adopted because of their added hyper-parameters.

3.2.8 MATPAC++: Enhanced Masked Latent Prediction for Self-Supervised Audio Representation Learning

3.2.8.1 Summary of the research item

Quelennec et al. [12] address the ambiguity of predicting masked content with multiple choice learning: the predictor produces several hypotheses for each masked patch, and the best one, selected with an annealed temperature [33], is trained and passed to the classification task of MATPAC. With a unified, statistically controlled protocol (same data splits, 95 % confidence intervals, Wilcoxon tests), MATPAC++ obtained an average linear-probing score of 75.7 over seven tasks, against 75.2 for MATPAC and 74.3 for M2D. When trained only on music (about 0.9 M Million Song Dataset samples and 0.18 M private web-radio samples), its average over eight music tasks rose from 47.2 to 47.9, a gain of only

0.7 points over the AudioSet-trained model. In the same table the AudioSet-trained M2D averaged 46.3 against 44.2 for MERT, whose scores were taken from its paper.

3.2.8.2 Critical analysis of the research item

MATPAC++ offers the most careful comparison available of general-audio against music-only pre-training under one protocol, since the same model was trained on both kinds of data and evaluated with the same data splits. It is also unusual in reporting 95 % confidence intervals and Wilcoxon significance tests rather than single scores, which makes its small differences interpretable. However, the work is an arXiv preprint still under review, and the web-radio portion of its music data is private, so the music-only model cannot be fully reproduced. The method itself has a cost: the additional hypotheses increase the size of the predictor, and the benefit is smaller on structured music than on diverse audio. Finally, the MERT numbers in its comparison were taken from the MERT paper and therefore come from a different protocol, so the advantage of the AudioSet-trained M2D over MERT reported in the same table should be read with care rather than as a controlled result.

3.2.8.3 Relationship to the proposed research work

This is the key motivating observation of the project: general-audio models are already strong on music, and large-scale music-only training adds little. The question this project asks is whether a small, cheap, music-specific step can recover a comparable increment. The general and music MATPAC++ models serve as reference models in the experimental design.

3.2.9 Codified Audio Language Modeling Learns Useful Representations for Music Information Retrieval (JukeMIR)

3.2.9.1 Summary of the research item

Castellon et al. [34] used the hidden states of Jukebox, a language model trained on codified (discretely encoded) audio from about one million songs, as input features for shallow probes. Averaged over four tasks (tagging, genre classification, key detection and emotion recognition), the representations were 30 % stronger than those of models pre-trained on tag prediction, with a particularly large advantage for key detection, and competitive with purpose-built systems on several tasks.

3.2.9.2 Critical analysis of the research item

JukeMIR gave the first clear demonstration that self-supervised pre-training on music audio yields better representations than supervised pre-training on tags, with a 30 % average improvement over tag-based

models across four tasks. It also exposed a blind spot of tag-pretrained models: key detection, which tags rarely describe, benefited most from modeling the audio itself. This makes the study an early argument for representations that capture pitch and harmony. However, the practical cost of the approach is very high. Jukebox has billions of parameters [7], so even feature extraction is expensive, and further adaptation of the model is impractical for most research groups. The model was also designed as a generative language model rather than as an encoder, so its usefulness as a feature extractor is a by-product rather than a design goal. Finally, the evaluation covers only four tasks, which is a narrow basis for the general claim that music language modeling learns useful representations.

3.2.9.3 Relationship to the proposed research work

The result that key detection benefits from modeling music audio, not tags, is one reason for adding a pitch-oriented teacher in this project. The cost of very large models is also the reason for starting from an 86 M-parameter encoder.

3.2.10 Supervised and Unsupervised Learning of Audio Representations for Music Understanding (MULE)

3.2.10.1 Summary of the research item

McCallum et al. [35] compared pre-training on music with pre-training on general audio (a version of AudioSet), and supervised with unsupervised (SimCLR-style) training, on tasks such as genre, era, mood, instrumentation, key and tempo. Supervised models on a large expert-annotated music catalog (about 1.8 M pieces, 117,497 hours) reached state-of-the-art results with under 100 M parameters but specialized to their labels. Among the unsupervised models, restricting the pre-training data to music allowed training with smaller batch sizes while achieving the best unsupervised results.

3.2.10.2 Critical analysis of the research item

MULE offers a controlled comparison of the effect of the pre-training domain, since music and general-audio data were compared under the same methods, both supervised and unsupervised. It also pays attention to model size, showing that models of fewer than 100 M parameters can reach state-of-the-art results without any fine-tuning, which keeps the conclusions relevant to practical systems. Its finding that restricting unsupervised pre-training to music allows smaller batch sizes while giving the best unsupervised results is a direct statement about the value of in-domain data. However, the large expert-annotated music catalog on which the supervised models were trained is not publicly available, so the central result cannot be reproduced or extended by others. The unsupervised models also use contrastive

training with large batches rather than masked latent prediction, so it is not certain that the same domain effect applies to the masked-prediction models that dominate current audio representation learning.

3.2.10.3 Relationship to the proposed research work

MULE gives direct evidence that the domain of the pre-training data matters, which the present study tests for a masked-latent-prediction model by adding a small amount of in-domain data rather than by replacing the data.

3.2.11 MERT: Acoustic Music Understanding Model with Large-Scale Self-Supervised Training

3.2.11.1 Summary of the research item

Li et al. [7] train a transformer with HuBERT-style masked prediction on raw audio and two kinds of targets. The acoustic teacher gives discrete labels, either from K-means on log-mel and chroma features or from the residual quantization codes of EnCodec [36]. The musical teacher is the constant-Q transform: a linear head regresses the CQT (84 bins, 12 per octave, from 32.7 Hz) of the audio with a mean-squared-error loss, and the total loss is $\alpha\mathscr{L}_H + \mathscr{L}_{\mathrm{CQT}}$. Short excerpts from the same batch are mixed into each clip (in-batch noise mixup). Models of 95 M parameters were trained on about 1,000 hours, and a 330 M-parameter model on 160,000 hours, with 5-s contexts. On 14 tasks evaluated by probing, MERT achieved the best overall score. Adding the CQT loss raised key-detection accuracy from 55.1 to 65.0 and GTZAN genre accuracy from 75.2 to 78.6 for the K-means variant, and from 60.5 to 63.2 and from 72.8 to 77.2 for the RVQ variant.

3.2.11.2 Critical analysis of the research item

MERT contributes a clear mechanism, the CQT musical teacher, that injects pitch and harmonic bias into the representation, and the ablation validates it: the CQT loss raised both key-detection and GTZAN accuracy for both teacher variants. The models are also competitive at only 95 M parameters, and the version trained on open data (MERT-v0-public, about 900 hours) shows that the recipe works with limited data. However, scaling the method proved difficult. Training at 330 M parameters was unstable and needed special normalization and attention relaxation, and all contexts are limited to 5 s, which restricts the modeling of longer musical structure. The strongest models also rely on a very large collection of music, so the best results are not reachable with open data alone. Finally, the released weights are licensed for non-commercial use only [11], which restricts their use as a base for derived models and as part of applications built on them.

3.2.11.3 Relationship to the proposed research work

MERT supplies the musical teacher: the framework adds a CQT-regression task with the same bin layout and a unit weight. MERT-v1-95M and MERT-v0-public are reference models, one with 20,000 hours and one with about 900 hours of training data. Its in-batch noise mixup corresponds to the background-noise mixing of M2D-X.

3.2.12 A Foundation Model for Music Informatics (MusicFM)

3.2.12.1 Summary of the research item

Won et al. [8] adapt BEST-RQ [37] to music: a frozen random projection and a random codebook turn the normalized log-mel features into discrete tokens, and a 12-layer Conformer is trained to predict the tokens of masked segments. They compared model sizes, token rates, input durations and data, and evaluated on token-level and sequence-level tasks such as beat tracking, chord and structure analysis, key detection and tagging. Longer (30-s) inputs improved tasks that need long-term context. A model trained on the whole Free Music Archive (about 8,000 hours) scored below the model trained on 160,000 hours of in-house data on most tasks, which the authors attribute to scale and to the crowd-sourced nature of FMA.

3.2.12.2 Critical analysis of the research item

MusicFM adapts BEST-RQ so that no external tokenizer is needed: a frozen random projection and codebook provide the targets, which removes a whole training stage from the pipeline. The authors also compare model size, token rate, input length and training data systematically, and their evaluation covers local (token-level) tasks such as beat tracking and chord recognition as well as global tasks such as tagging. However, the random targets carry no musical prior, so any musical knowledge must emerge from the data alone, and the model is large, with about 330 M parameters. The most relevant weakness concerns the data source. Training on the whole Free Music Archive, about 8,000 hours, gave clearly weaker results on most tasks than training on 160,000 hours of curated in-house data, which the authors attribute both to scale and to the crowd-sourced nature of FMA. This indicates that FMA alone is a limited pre-training source, even when the whole archive is used.

3.2.12.3 Relationship to the proposed research work

MusicFM shows what training from scratch on the whole FMA achieves, and thereby sets a ceiling for the data source used here. Its FMA model is a reference model, and the observation that FMA is a weaker pre-training source than a larger private corpus is a caution that applies to this project.

3.2.13 MuQ: Self-Supervised Music Representation Learning with Mel Residual Vector Quantization

3.2.13.1 Summary of the research item

Zhu et al. [9] train a 12-layer Conformer to predict tokens from a Mel residual vector quantizer (Mel-RVQ). The quantizer has a single linear layer as encoder and decoder and a residual codebook structure, is trained on the mel spectrum, and replaces both the random projection of BEST-RQ and the large codec of MERT. Pre-trained on only 0.9K hours of the open Music4All dataset, MuQ already outperformed MERT and MusicFM on the MARBLE benchmark (average 75.8 against 74.4 and 75.4); with 160K hours of in-house data it reached 76.7, and with iterative refinement 77.0. A music-text variant, MuQ-MuLan, achieved a state-of-the-art zero-shot tagging score on MagnaTagATune.

3.2.13.2 Critical analysis of the research item

MuQ shows that a good definition of the prediction target can matter more than the volume of data: with only 0.9K hours of open Music4All audio, it already outperformed MERT and MusicFM on the MARBLE benchmark. Its Mel-RVQ quantizer is also tiny and simple to train, with a single linear layer as encoder and decoder and a residual codebook, so it avoids both the random targets of BEST-RQ, which carry no musical prior, and the large codec used by MERT. However, the best MuQ models still rely on 160,000 hours of private data and a large multi-GPU budget, so the strongest results cannot be reproduced with open resources. The evaluation is by linear probing on a limited set of tasks, which leaves open how the representations behave under other protocols. In addition, the tokenizer needs a separate training stage before pre-training, so the pipeline remains more complex than methods that predict continuous targets directly.

3.2.13.3 Relationship to the proposed research work

MuQ provides further evidence that a small amount of music can suffice if the learning signal is well chosen, which is the premise of the musical teacher. It is an alternative to CQT targets, but it would require training a quantizer and pre-training from scratch.

3.2.14 PupuJEPA / PupuM2D: Frequency-Aware Self-Supervised Music Representation Learning

3.2.14.1 Summary of the research item

Gu et al. [10] propose a masked-latent-prediction model trained directly on 2-D music spectrograms. It was first called PupuJEPA and renamed PupuM2D in the third version of the preprint, because the

target encoder, as in M2D, receives only the masked patches. Domain-specific changes include 4×16 patches that give a 25 Hz frame rate, rotary position embeddings, SwiGLU feed-forward layers, QK-normalization, a smoothed-L1 loss, a mixture of random, blockwise and time-frequency masking (overall ratio 0.8) whose proportions follow a curriculum, and the removal of DropPath, LayerScale and batch normalization, which caused collapse. It was trained on about 100,000 hours of in-house music in five sizes from 5 M to 632 M parameters. On the MARBLEv2 benchmark with linear probing, PupuM2D-Large reached 66.1 on GiantSteps key detection, 86.9 on GTZAN genre and 40.8 AP on MagnaTagATune. In the same tables, AudioSet-trained checkpoints evaluated by the authors were at or above the music-specific MERT, MuQ and MusicFM on tagging and instrument classification: for example, MagnaTagATune AP was 40.9 for M2D, 40.9 for an M2D-X checkpoint and 41.7 for MATPAC++, against 37.5 to 38.5 for MERT, MuQ and MusicFM, and the MTG-Jamendo instrument AP was 21.1 for M2D and MATPAC++ against 18.5 to 19.7 for the music models. MATPAC++ also matched or exceeded PupuM2D-Large on genre and tagging.

3.2.14.2 Critical analysis of the research item

The PupuM2D study is valuable because it evaluates M2D, M2D-X and MATPAC++ next to the music-specific MERT, MuQ and MusicFM in one environment, and because it re-trains three audio baselines on the authors' music data under identical settings, which separates the effect of the data from that of the architecture. Its ablations of architecture, masking and inference paradigm are thorough, and the attention maps are presented by the authors as showing musically meaningful patterns, although this interpretation is qualitative. However, the work is a preprint that has not yet been peer reviewed, and it has already been revised, including a change of the model's name. It was trained on a private corpus of 100,000 hours with a very large compute budget across five model sizes, so it cannot be reproduced, and its models cannot be compared with smaller ones at equal cost. Its main conclusions must therefore be accepted largely on the strength of the authors' own evidence.

The reported numbers also need careful reading. The scores of the 2-D models are the best over several swept inference paradigms, which can make them optimistic compared with a single fixed protocol. The results are reported on MARBLEv2, whose scores are not strictly comparable with those of MARBLE v1, so they cannot be placed directly next to earlier published numbers. Most importantly, the advantage of a 100,000-hour music model over strong general-audio models is clear mainly for valence in emotion recognition. For tagging, genre and instrument classification it is small or absent: the AudioSet-trained M2D, M2D-X and MATPAC++ checkpoints matched or exceeded MERT, MuQ and MusicFM on MagnaTagATune and MTG-Jamendo, and MATPAC++ matched or exceeded PupuM2D-Large on genre and tagging. Key detection and beat tracking, where a music model might be expected to help most, could

not be compared at all, because the frame rate of the general models is too low.

3.2.14.3 Relationship to the proposed research work

This is the most recent evidence for the central question. It shows that the M2D design also works for music-specific pre-training, and it quantifies how little a 100,000-hour in-domain model adds over AudioSet-trained models on tagging, genre and instrument tasks, while frame-level tasks such as key detection and beat tracking are where a music-specific model is most useful. These are the tasks that the musical teacher of MERT targets. The study also warns about the cost of fine temporal resolution: the proposed work keeps 16×16 patches (160 ms per step) and therefore excludes frame-level tasks from its evaluation.

3.2.15 Robust wav2vec 2.0: Analyzing Domain Shift in Self-Supervised Pre-Training

3.2.15.1 Summary of the research item

Hsu et al. [38] studied what happens when the unlabeled pre-training data, the labeled fine-tuning data and the test data come from different domains. Using target-domain data during pre-training led to large improvements, and in a large-scale setup pre-training on unlabeled in-domain data reduced the gap between models trained on in-domain and on out-of-domain labeled data by 66 % to 73 %. Pre-training on several domains also improved generalization to domains that were not seen, and continual pre-training (first one domain, then another) performed similarly to joint training on the combined data when more in-domain data were added.

3.2.15.2 Critical analysis of the research item

This study offers a large-scale, systematic analysis of data-domain mismatch in self-supervised speech models, varying the domains of the pre-training, fine-tuning and test data separately. Its practical message is strong: unlabeled target-domain data are much easier to obtain than labeled data, and adding them during pre-training closed most of the gap between in-domain and out-of-domain labeled training. The finding that continual pre-training performed similarly to joint training is also encouraging for adapting an existing model. However, the setting differs from music representation learning in several ways. The task is speech recognition, the objective is contrastive rather than masked latent prediction, and the corpora contain hundreds to thousands of hours, far more in-domain data than the 83 hours used here. More importantly, the study measures accuracy on the target domain and on unseen domains but not the loss of ability on the original domain, so it gives no information about forgetting.

3.2.15.3 Relationship to the proposed research work

It supports the premise that a small amount of unlabeled in-domain audio can close part of a domain gap, and the benefit of multi-domain pre-training agrees with the choice to keep general-audio data (the background noise) in the training mix.

3.2.16 Self-Distillation for Further Pre-training of Transformers

3.2.16.1 Summary of the research item

Lee et al. [39] observed that further pre-training of vision transformers on small target data overfits. Their remedy first further pre-trains the initial model on the target unlabeled data to obtain a teacher, then takes the same initial model as a student and trains it with the masked-autoencoding objective while pulling its hidden representations towards those of the teacher. The method outperformed the baselines on several image and text classification benchmarks.

3.2.16.2 Critical analysis of the research item

The method of Lee et al. directly targets the overfitting that occurs when a transformer is further pre-trained on a small dataset, and it does so by regularizing in representation space rather than by changing the pretext task. Because it was shown to work for both vision and language transformers, it suggests that the problem and its remedy are not specific to one modality. However, the procedure requires two rounds of further pre-training, first to obtain the teacher and then to train the student, which doubles the cost of adaptation. The evaluation is also restricted to image and text benchmarks, so its effect on audio has not been tested. A more fundamental concern is the choice of teacher. The teacher is itself a model further pre-trained on the same small target data, and it may therefore carry the overfitting that the method is meant to remove, which limits how far the student can be protected from it.

3.2.16.3 Relationship to the proposed research work

M2D-X takes the opposite choice of teacher: the frozen original model, which costs no extra training round and cannot overfit the new data. The two approaches share the idea that representation-level regularization is the right remedy.

3.2.17 SONAR: Self-Distilled Continual Pre-training for Domain Adaptive Audio Representation

3.2.17.1 Summary of the research item

Zhang et al. [14] continue pre-training BEATs on unlabeled data from four domains (speech emotion, music from FMA-Large, bioacoustics and environmental sound), with 30,000 to 50,000 audio segments per domain. The framework combines task-relevant stratified sampling from a general pool and the domain data, dual-source self-distillation that constrains both the tokenizer and the encoder to stay close to the frozen original, and an online-clustered codebook that adapts the tokenizer to new acoustic patterns. For music, direct continual pre-training that resumes the original objective lowered the frozen-feature GTZAN F1 score from 86.0 to 76.0 and the AudioSet tagging mAP from 34.8 to 14.7, a forgetting rate of 57.8 %; SONAR raised GTZAN to 88.0 and kept the mAP at 34.7.

3.2.17.2 Critical analysis of the research item

SONAR is the first audio study to quantify forgetting after domain adaptation, and it includes a music case built on FMA together with a regularized method. Its results are striking: direct continual pre-training lowered the frozen-feature GTZAN score from 86.0 to 76.0 and the AudioSet mAP from 34.8 to 14.7, while distillation towards the frozen original model gave near-zero forgetting and also improved GTZAN. However, the work is an arXiv preprint, and it is built on the discrete-token model BEATs, so several of its components, such as the self-distillation of the tokenizer and the online-clustered codebook, do not apply to M2D. The regularization weight is very large (10^6) and was tuned only once, which leaves open how sensitive the results are to it. The evaluation also uses a single downstream task per domain, so the music result is judged by GTZAN alone, a small dataset with known faults.

3.2.17.3 Relationship to the proposed research work

SONAR is the closest prior study: it uses FMA, unlabeled data, a small adaptation set and a frozen-original regularizer. This project differs in the base model (M2D, continuous targets), in adding a musical teacher, in the single-GPU budget, and in a larger and more varied set of music tasks. Its forgetting numbers also confirm that naive continuation must be treated as a control that may fail.

3.2.18 Audio Set: An Ontology and Human-Labeled Dataset for Audio Events

3.2.18.1 Summary of the research item

AudioSet [5] is an ontology of audio events and a collection of human-labeled 10-s segments from YouTube videos. Its project page lists 2,084,320 labeled clips, about 5.8 thousand hours, and an ontology

of 632 event classes, of which 527 are annotated with clips. M2D was pre-trained on the 2,005,132 clips that were still downloadable, which amount to 5,569 hours [4].

3.2.18.2 Critical analysis of the research item

AudioSet is the largest labeled general-audio collection and has become the standard pre-training corpus for audio self-supervision, so most of the general-audio models in this review were trained on it. Its ontology of 632 classes and more than two million clips give it considerable breadth, and it contains a substantial amount of music. However, only links to online videos are distributed, not the audio itself. As videos are removed, clips disappear, and every group ends up training on a different subset; M2D, for example, was trained on the 2,005,132 of the 2,084,320 listed clips that were still downloadable. This makes published numbers hard to compare [12], because a difference between two models may come partly from their data. In addition, the audio quality and the labels are of web-video quality, and the music portion is not curated, so there is no control over the styles or recording conditions of the music that a model sees.

3.2.18.3 Relationship to the proposed research work

AudioSet is the source of the general-audio knowledge in the M2D checkpoint. It is not downloaded in this project, which relies on the released weights.

3.2.19 FMA: A Dataset for Music Analysis

3.2.19.1 Summary of the research item

Defferrard et al. [13] released the Free Music Archive dataset: 106,574 tracks by 16,341 artists in a hierarchy of 161 genres, with full-length audio of the order of 900 GiB under Creative Commons licenses, plus metadata and pre-computed features. Subsets are provided as 30-s clips cut from the middle of the tracks: small (8,000 clips, 8 balanced genres), medium (25,000 clips, 16 genres, selected for metadata completeness and popularity) and large (all tracks). An 80/10/10 split into training, validation and test sets is stratified by genre and filtered by artist so that no artist appears in two splits.

3.2.19.2 Critical analysis of the research item

FMA stands out among music datasets because it provides redistributable full-quality audio under Creative Commons licenses, together with artist-level metadata and training, validation and test splits that are filtered by artist to prevent leakage. It also comes in several sizes, from the small subset, comparable in size to GTZAN, to the whole archive of more than 100,000 tracks, so it can serve both as an evaluation

set and as a pre-training corpus. However, the genre labels are provided by the artists themselves and are therefore noisy, and some genres, such as experimental music, are under-represented in the medium subset. Documented issues include corrupted or truncated tracks, tracks shorter than 30 s and duplicates [40], so the audio must be audited before use. Finally, models trained only on FMA trail models trained on a larger curated corpus [8], which the MusicFM authors attribute partly to the crowd-sourced nature of the archive.

3.2.19.3 Relationship to the proposed research work

FMA is the further pre-training corpus: 10,000 tracks of the medium subset, taken from the training split only. Its FMA-small test split is the in-domain genre evaluation. The sampling, audit and conversion are specified in Chapter 5.

3.2.20 FSD50K: An Open Dataset of Human-Labeled Sound Events

3.2.20.1 Summary of the research item

Fonseca et al. [15] released FSD50K, 51,197 Freesound clips (108.3 hours) labeled with 200 classes of the AudioSet ontology. The clips are divided into a development and an evaluation split without common uploaders, the evaluation set is exhaustively labeled, and all clips are under Creative Commons licenses so that the waveforms can be freely redistributed.

3.2.20.2 Critical analysis of the research item

FSD50K provides openly licensed audio, so its waveforms can be freely redistributed, unlike AudioSet, which distributes only links to videos. Its labels follow the AudioSet ontology, its development and evaluation splits share no uploaders, and its evaluation set is exhaustively labeled, which makes it a reliable and well-documented source of everyday sounds. A further practical advantage is that it is already used as the background-noise corpus in the M2D-X recipe, so it has been tested in exactly that role. However, the dataset is of moderate size, 51,197 clips and 108.3 hours, and the clips are short and of variable length, so the variety of available noise is limited and long noise segments cannot always be cut from a single clip. More importantly for music applications, its 200 classes include musical-instrument classes, so it is not purely non-musical noise. When it is mixed into music as a distractor, part of the added noise may itself be music.

3.2.20.3 Relationship to the proposed research work

The development portion is the noise corpus of the denoising task. An optional run removes music-related classes to test whether purely non-musical noise is a better distractor.

3.2.21 Music4All: A New Music Database and Its Applications

3.2.21.1 Summary of the research item

Santana et al. [41] built Music4All, a database of 109,269 pieces with 30-s audio excerpts, about 910 hours in total, together with song metadata. An extension (Music4All-Onion) adds audio, video and metadata features and listening records of about 119,000 users. Music4All is the corpus on which MERT-v0-public (together with part of FMA) and the small-data version of MuQ were trained.

3.2.21.2 Critical analysis of the research item

Music4All is a moderately sized, quality-controlled corpus of 30-s music excerpts, about 910 hours in total, and it is the main open alternative to FMA for training music models. It is accompanied by rich side information, including song metadata and, through the Music4All-Onion extension, audio, video and metadata features and the listening records of about 119,000 users, which makes it useful for recommendation as well as for representation learning. Its value for pre-training is shown by its use for MERT-v0-public and for the small-data version of MuQ. However, the collection consists mostly of popular music with limited stylistic diversity, as noted by the MERT authors, so models trained on it may see little of less mainstream styles. The audio is also limited to 30-s excerpts rather than complete tracks, so the long-term structure of a piece, such as the relation between its sections, cannot be learned from it.

3.2.21.3 Relationship to the proposed research work

Music4All is the data behind the small-data music-specific references. Its scale of about 910 hours is the natural comparison for the 83 hours used here. It is not used for pre-training because FMA offers artist-filtered splits and open licenses.

3.2.22 MARBLE: Music Audio Representation Benchmark for Universal Evaluation

3.2.22.1 Summary of the research item

Yuan et al. [16] proposed MARBLE, a benchmark of music representations built on a taxonomy of four levels (acoustic, performance, score and high-level description). It defines a unified protocol of 18

tasks on 12 public datasets, with frozen pre-trained models and simple downstream heads, and provides baselines for the open-source models pre-trained on music recordings; large-scale pre-trained musical language models performed best on most tasks, with room for improvement. Its tasks include key detection, emotion regression, instrument and mood tagging on MTG-Jamendo and beat tracking, and three of its datasets are used in this project. GTZAN [42] consists of 1,000 excerpts of 30 s in ten genres; Sturm [43] catalogued its faults (exact, recording and version repetitions, repeated artists, mislabeled and distorted excerpts), and Kereliuk et al. [18] proposed a fault-filtered split with 443 training, 197 validation and 290 test clips. MagnaTagATune [44] consists of 25,863 clips cut from over 5,000 songs, with tags collected through the TagATune game, distributed as 16 kHz mono MP3 files; the standard task is tagging with the 50 most frequent tags. NSynth [45] contains 305,979 monophonic notes of 4 s from 1,006 instruments, annotated with pitch, velocity and one of 11 instrument families.

3.2.22.2 Critical analysis of the research item

MARBLE brings broad coverage to the evaluation of music representations: tagging, key detection, genre, emotion, instrument, pitch, beat tracking and structure tasks are all assessed under one protocol, with frozen models, simple downstream heads and a common leaderboard. Because its datasets are widely used, results can be compared with MERT, MuQ, MATPAC and earlier work, and NSynth in particular gives exact pitch labels. The taxonomy of acoustic, performance, score and high-level description levels also helps to show which kinds of musical knowledge a model has acquired. However, the breadth of the benchmark has a cost. Many tasks need large downloads and several downstream heads, which is expensive for a project with a single GPU. The implementation has also been revised, and later papers state that scores of the revised version are not strictly comparable with those of the original [10], so the version behind every published number must be checked before results are compared.

Beyond the benchmark as a whole, the individual datasets used in this project have known problems that affect how their results can be read. GTZAN contains exact, recording and version repetitions, repeated artists, and mislabeled and distorted excerpts, as catalogued by Sturm, so results on random splits are optimistic. The fault-filtered split of Kereliuk et al. removes these problems, but its test set has only 290 clips, so differences of a few points between models may not be meaningful. MagnaTagATune is distributed as low-quality 16 kHz mono MP3 audio and contains several clips cut from each song, so any split must keep the clips of one song together to avoid leakage between training and test data. NSynth offers precise pitch and instrument labels, but it consists of isolated monophonic notes, far from polyphonic music, so good results on it do not guarantee that a model can recognize pitch or instruments in real recordings.

3.2.22.3 Relationship to the proposed research work

MARBLE guided the choice of music tasks. GTZAN genre (with the fault-filtered split), MagnaTagATune tagging, and NSynth instrument and pitch classification are part of the evaluation of Chapter 5, and they are also the four tasks demonstrated by the dashboard of Chapter 4. The project evaluates them with the linear-probe toolkit of the M2D authors [20] for budget reasons and does not mix its numbers with MARBLE scores; because of the small GTZAN test split, no conclusion rests on GTZAN alone.

3.2.23 HEAR: Holistic Evaluation of Audio Representations

3.2.23.1 Summary of the research item

Turian et al. [6] report the HEAR 2021 challenge, which asked which audio embedding approach generalizes best to a wide range of downstream tasks across everyday domains without fine-tuning. Twenty-nine models submitted by thirteen external teams were evaluated through a common application programming interface on nineteen tasks derived from sixteen datasets covering speech, environmental sound and music.

3.2.23.2 Critical analysis of the research item

HEAR was the first large, open comparison of general-purpose audio embeddings under one protocol: twenty-nine models from thirteen teams were evaluated through a common interface on nineteen tasks from sixteen datasets, without fine-tuning. This design made it possible to compare very different models fairly, and it established frozen-feature evaluation across many tasks as a standard way of judging audio representations. The challenge also had a lasting influence on the field, and the M2D line of work began as a submission to it [26]. However, the evaluated models date from 2021 and do not include current masked-latent-prediction models such as M2D, MATPAC and their successors, so its rankings no longer describe the state of the art. Music is also only one of several domains, and only some of its tasks involve music, so a model's HEAR results give limited information about its musical ability, which a music-focused benchmark is needed to assess.

3.2.23.3 Relationship to the proposed research work

HEAR establishes the principle of judging representations by frozen features on many tasks, which this project applies to a music-focused task set.

3.3 Literature Review Summary Table

Table 3.1 summarizes the reviewed research studies, datasets and benchmarks: the author and reference, the method or content, the main reported result, and the main limitation with respect to this project.

Table 3.1: Summary of the reviewed literature: the author and reference, the method or content, the main reported result, and the main limitation with respect to this project.

Author	Method	Results	Limitations
Huang et al. [24]	Masked autoencoder on spectrograms; 80% masking; local-window decoder	AudioSet-2M mAP 47.3; ESC-50 94.1% (fine-tuned)	Reconstruction target; needs fine-tuning
Chen et al. [25]	Iterative acoustic tokenizer and discrete-label prediction	AudioSet-2M mAP 48.6 (single), 50.6 (ensemble); ESC-50 98.1%	Multi-stage; best variants use a supervised teacher
Niizumi et al. [3]	Online and target networks; target sees only masked patches; ViT-Base on AudioSet	Linear-evaluation average 80.2 on nine tasks; best on four	Costly pre-training; less sensitive to pitch
Niizumi et al. [4]	Background noise and an offline network (labels, distillation or regularization)	ICBHI score 61.93, 61.17 (naive), 62.73 (noise), 63.29 (M2D-X)	One small medical task; no music; forgetting not measured
Riou et al. [28]	Study of masking and input duration for audio JEPA	Random masking best; GTZAN 82.1 to 86.9 as duration grows from 2.1 to 6.4 s	AudioSet only; few designs tested
Tuncay et al. [29]	Audio JEPA, ViT-Base, AudioSet, about 13 epochs	kNN: first on ESC-50, FMA-small and GTZAN among three models	Under-trained; weak linear probing; preprint
Quelennec et al. [31]	Latent prediction plus DINO-style classification	Average 74.9 against 74.0 without classification and 73.7 for M2D; MTT 41.1 mAP	Classification acts as regularizer; AudioSet version drift
Quelennec et al. [12]	Multiple-hypothesis predictor with annealed MCL	Average 75.7; music-only 47.9 against 47.2 for AudioSet training	Preprint; private data
Castellon et al. [34]	Probes on Jukebox language-model features	30% better than tag-pretrained models on four tasks; strong key detection	Billions of parameters; four tasks
McCallum et al. [35]	Music against general audio; supervised against contrastive	Music-only pre-training best among unsupervised models, with smaller batches	Private 117,497-h dataset
Li et al. [7]	Masked prediction with acoustic and CQT teachers; 95M and 330M	Best overall on 14 tasks; key 55.1 to 65.0 with CQT	5-s context; instability; large data
Won et al. [8]	Random-projection targets; Conformer, 330M	FMA-only (8,000 h) below the 160,000-h model	Random targets; large model
Zhu et al. [9]	Mel-RVQ tokens; Conformer	0.9K h beats MERT and MusicFM; average 75.8, 76.7, 77.0	160K-h data private; separate tokenizer stage

Author	Method	Results	Limitations
Gu et al. [10]	2-D music model, 100,000 h, mixed masking, SwiGLU, RoPE	Key 66.1; GTZAN 86.9; MTT AP 40.8; general models match on tagging and instruments	Preprint; private data; very large compute
Hsu et al. [38]	Pre-training on in-domain unlabeled speech	Closes 66–73% of the domain gap	Speech; large corpora
Lee et al. [39]	Self-distillation for further pre-training of transformers	Beats baselines on image and text benchmarks	Two rounds; teacher may overfit
Zhang et al. [14]	Continual pre-training of BEATs with distillation and sampling	GTZAN F1 86.0 (original), 76.0 (naive), 88.0 (SONAR); AudioSet mAP 34.7 against 14.7	Preprint; tokenizer-specific
Gemmeke et al. [5]	Dataset: 2,084,320 labeled 10-s YouTube clips; 527 annotated classes	Standard general-audio pre-training corpus; source of M2D	Clips disappear; versions differ
Defferrard et al. [13]	Dataset: 106,574 Creative Commons tracks; subsets of 8,000 and 25,000 clips	Artist-filtered splits; redistributable audio	Noisy genre labels; some corrupted files
Fonseca et al. [15]	Dataset: 51,197 Freesound clips; 200 classes	Openly licensed; noise corpus of M2D-X	Short clips; includes instrument classes
Santana et al. [41]	Dataset: 109,269 excerpts of 30 s; about 910 h	Training data of MERT-v0-public and small MuQ	Mostly pop; 30-s excerpts
Yuan et al. [16]	Benchmark: 18 tasks on 12 datasets, including GTZAN, MagnaTagATune and NSynth	Large musical language models best on most tasks	Heavy; versions not comparable; dataset faults
Turian et al. [6]	Benchmark: 29 models, 19 tasks, 16 datasets	First broad frozen-embedding comparison	2021 models; few music tasks

3.4 Conclusion

The literature shows that masked latent prediction on spectrograms yields strong general-purpose audio representations, with M2D, MATPAC and MATPAC++ at the top of the AudioSet-trained models. These general-audio models are already strong on music: in the unified comparison of MATPAC++, the AudioSet-trained models were on a par with or above MERT, and music-only training added about 0.7 points on average [12]; in the evaluation of PupuM2D, AudioSet-trained M2D, M2D-X and MATPAC++ checkpoints matched or exceeded MERT, MuQ and MusicFM on tagging and instrument classification, and a 100,000-hour in-domain model was clearly ahead mainly on valence in emotion recognition, while the frame-level tasks (key, beat) could not be compared with the general models [10]. Music-specific models gain from modeling music audio and from music-oriented targets, such as the CQT of MERT and the Mel-RVQ tokens of MuQ [34, 35, 7, 9], but they rely on 900 to 160,000 hours of music, often private, and a model trained on FMA alone was weaker than one trained on a larger curated corpus [8].

Further pre-training on domain data closes part of a domain gap [38], and the M2D-X recipe makes it

practical on one GPU through background noise and a frozen-teacher regularizer [4]. The risks are also documented: further pre-training on small data can overfit [39], and naive continuation can forget, as the FMA-based experiments of SONAR on a different model family show [14]. Open data suffice to run such a study: FMA is the only open, artist-filtered and redistributable full-quality music corpus of suitable size, FSD50K supplies noise, and MARBLE provides the music tasks, while HEAR established the frozen-feature protocol. Protocols differ between benchmarks, so numbers from different papers cannot be compared directly.

No reviewed work applies the M2D-X recipe to music, combines noise, a frozen-teacher regularizer and a music-specific CQT teacher in one framework, or does so on a small open subset of about 10,000 tracks with one GPU. This project addresses that gap: it further pre-trains the AudioSet M2D model on FMA with the M2D-X-Music objective, compares each component in a controlled set of further pre-training runs, and compares the result with music-specific references.

Chapter 4 Software Requirement Specifications

This chapter specifies the software that accompanies the research: a small web dashboard, called the M2D Music Dashboard in this report, that is built for demonstration only. The dashboard has two purposes. First, it presents the results of the further pre-training study of Chapter 5 in a form that a supervisor, an evaluator or another student can explore without reading the result files. Second, it demonstrates four music downstream tasks that belong to the MARBLE benchmark [16], namely genre classification (GTZAN [42]), music auto-tagging (MagnaTagATune [44]), instrument classification and pitch classification (both NSynth [45]). A visitor chooses a task, gives an audio clip and sees the prediction of the original or the further pre-trained M2D model. The training and evaluation pipeline itself, the datasets and the models are specified in Chapter 5 and are not repeated here; this chapter only covers the web application and its development. The dashboard is deliberately simple and minimal: it has no user accounts, no login and no sign-up. The chapter lists the features, the functional requirements, the quality attributes and the non-functional requirements, the assumptions, the use cases, the hardware and software requirements, the planned user interface, the database design and the risks.

4.1 List of Features

The dashboard is a single-page web application with three views: Results, Demo and About. The following features will be available.

- The aggregate music score S_{mus} (Section 5.5) of every model, a table of the per-task scores with their spread over probe seeds, and a bar chart, with a filter by model group (original M2D, further pre-trained M2D and reference models).
- Four task cards (genre, tagging, instrument and pitch) that show the dataset, the label set and the metric of each task.
- Upload of a WAV or MP3 file, of which the first 6 s are used, or selection of a bundled, openly licensed 6-s sample clip, with in-browser playback and a log-mel spectrogram preview.
- A choice between the original M2D checkpoint and the further pre-trained M2D.
- The top-5 genres with probabilities; the tags above the decision threshold together with the ten highest tag scores; the top-3 instrument families; and the top-3 pitches as MIDI number, note name and frequency. The inference time is shown with every prediction, and the label given by the source dataset is shown for sample clips.
- The same clip can be run through both models and the two outputs are shown side by side, with

the labels predicted by only one model highlighted.

- A short description of the project, the models, the datasets and their licenses, and a statement that single-clip predictions are a demonstration, not benchmark results.

4.2 Functional Requirements

The functional requirements describe the external behavior of the dashboard. Three kinds of users are involved: the Visitor, who uses the web interface (supervisor, evaluator, researcher or student); the Developer, who maintains the dashboard and loads results through a command-line script, since there is no administration page; and the System itself for automatic behavior. Table 4.1 lists the requirements.

Table 4.1: Functional requirements of the dashboard. Each requirement has an identifier, a statement and the user to whom it applies.

ID	Requirement	User
FR-01	The system shall open on the Results Overview and show the music score S_{mus} for every model stored in the database.	Visitor
FR-02	The system shall show a table of per-task test scores (mean and standard deviation over probe seeds) with the metric of each task.	Visitor
FR-03	The visitor shall be able to filter the results by model group: the original M2D, the further pre-trained M2D and the reference models.	Visitor
FR-04	The system shall draw a bar chart of the mean music score of the selected models.	Visitor
FR-05	The system shall list the four demonstration tasks with their dataset, number of labels and metric, and the visitor shall be able to select one.	Visitor
FR-06	The visitor shall be able to upload one audio file in WAV or MP3 format of at most 10 MB.	Visitor
FR-07	The visitor shall be able to select a bundled sample clip instead of uploading a file, and to play the chosen clip in the browser.	Visitor
FR-08	The system shall check the type, size and readability of an uploaded file and reject an invalid file with an error message that states the reason.	System
FR-09	The system shall convert the input to mono at the sampling rate of the selected model (16 kHz for M2D), keep at most the first 6 s, compute the log-mel spectrogram with the normalization used in evaluation, and show a preview of it.	System
FR-10	The visitor shall be able to choose between the original M2D and the further pre-trained M2D for the prediction; the further pre-trained M2D shall be selected by default.	Visitor
FR-11	The system shall compute the clip embedding with the frozen encoder from one 6.08-s input window, padded if the clip is shorter, and apply the linear probe of the selected model and task.	System
FR-12	The system shall display the task-specific output: top-5 genres; tags whose probability exceeds the threshold of the probe and the ten highest tag scores; top-3 instrument families; top-3 pitches with MIDI number m, note name and frequency $440 \cdot 2^{(m-69)/12}$ Hz. It shall also display the inference time, the device used and, for sample clips, the source label.	Visitor
FR-13	The visitor shall be able to run both models on the same clip and see the two outputs side by side, with the labels predicted by only one model highlighted.	Visitor
FR-14	The system shall queue inference requests when the model is busy and show the position of the request in the queue.	System

ID	Requirement	User
FR-15	The system shall delete an uploaded file as soon as its prediction has been returned.	System
FR-16	The system shall record anonymous metadata of every prediction (task, model, input source, duration, device, latency and status) in the inference log, without audio or network addresses.	System
FR-17	The developer shall be able to import evaluation results and register models, probes and sample clips with a command-line script that validates every record before writing it.	Developer
FR-18	The system shall provide an About page with the project description, the models, the datasets and licenses, and a demonstration disclaimer.	Visitor

4.3 Quality Attributes

The following quality attributes guide the design and will be used to judge whether the dashboard serves its demonstrative purpose. Because the dashboard is a companion to a research study, the attribute that matters most is that it shows the research faithfully.

4.3.1 Usability

The audience includes evaluators who are not specialists in music information retrieval. The interface therefore uses one page with three tabs and a three-step flow on the Demo tab (choose a task, give audio, run), plain labels such as “Genre” rather than dataset codes, and short help texts beside each input. A first-time visitor should be able to obtain a prediction without instructions.

4.3.2 Correctness and Consistency

A demonstration is useful only if it reflects the models that were evaluated. The dashboard shall reuse the preprocessing, windowing and embedding code of the evaluation pipeline (Section 5.5) and load the same linear probes, rather than re-implementing them. The scores on the Results tab shall come from the same result files as the tables in the report, imported unchanged, so that the dashboard can never show a number that the report does not.

4.3.3 Reliability

The dashboard will be shown during evaluations and presentations, where a failure is costly. Invalid inputs shall be rejected with clear messages instead of errors, an inference failure shall not stop the server, and the Results tab shall remain usable even if the model cannot be loaded.

4.3.4 Maintainability and Extensibility

New checkpoints are produced during the project, and the further pre-trained M2D shown by the dashboard is chosen only at the end of the experiments. Replacing a model or adding a task shall therefore require only a probe file and new rows in the database, imported with the developer script, without any change to the web code. The backend separates the inference module, the database layer and the web interface so that each can be changed alone.

4.3.5 Portability

The dashboard shall run on the research workstation and on any machine with a recent Python environment, with or without an NVIDIA GPU, and the frontend shall need nothing more than a web browser.

4.4 Non-Functional Requirements

The non-functional requirements state measurable targets for the dashboard. The figures below are design targets for a demonstration on one workstation, not measured results.

4.4.1 Performance

The performance requirements of the dashboard are as follows.

- For a clip of up to 6 s, the system shall return a prediction within 5 s on the GPU host and within 30 s on CPU, excluding upload time.
- The Results Overview shall load within 2 s on the local network.
- The models and probes shall be loaded once when the server starts, not for every request.

4.4.2 Capacity and Concurrency

The capacity and concurrency requirement of the dashboard is as follows.

- Inference requests shall be processed one at a time by a single worker on the GPU, and further requests shall wait in a queue, so that a burst of requests cannot exhaust the GPU memory.

4.4.3 Availability

The availability requirements of the dashboard are as follows.

- The dashboard shall be available on the local network during the scheduled evaluation and presentation sessions; it is not intended as a permanent public service.

- The server shall start with one command and be ready within two minutes.

4.4.4 Security and Privacy

The security and privacy requirements of the dashboard are as follows.

- The system shall not create user accounts and shall not store personal data or tracking cookies.
- Uploaded files shall be stored under random names in a temporary directory and deleted after the prediction, or at the latest after ten minutes.
- The server shall enforce the file type and the size limit itself and shall not rely on checks in the browser.
- The web interface shall have read-only access to the result tables; results can be written only through the developer script.

4.4.5 Reusability

The reusability requirements of the software are as follows.

- The inference module (audio loading, embedding and probe prediction) shall be a Python package that can be used without the web server, for example from a notebook.
- The frontend shall obtain all data through a small JSON API, so that another interface could use the same backend.

4.4.6 Compatibility

The compatibility requirement of the dashboard is as follows.

- The frontend shall work in current versions of Google Chrome, Mozilla Firefox and Microsoft Edge at a desktop or laptop screen width.

4.5 Assumptions

The specification is based on the following assumptions.

- The further pre-trained M2D checkpoint and the linear probes of both models for the four demonstration tasks are produced by the evaluation of Chapter 5 before the dashboard is finalized.
- The dashboard is hosted on the research workstation with the RTX A4000 GPU, or on a CPU-only machine with slower predictions.

- The inputs are music recordings. The instrument and pitch probes are trained on single isolated notes from NSynth, so their outputs are meaningful only for similar inputs; for other music they are shown for illustration.
- Visitors have basic computer skills and a current web browser, and, to play clips, speakers or headphones.
- The result files imported into the dashboard are the final values produced by the evaluation pipeline.
- Visitors understand that a prediction on one clip illustrates a model and is not a measure of its quality; the benchmark scores are those on the Results tab.

4.6 Use Cases

The use cases below cover the central behavior of the dashboard. Figure 4.1 shows the use-case diagram: the Visitor takes part in UC-01 to UC-05 through the web interface, and the Developer takes part in UC-06 through the command line. UC-05 extends UC-04 by running a second model on the same clip. Tables 4.2 to 4.7 describe each use case in the standard format, with step numbers that alternate between the actor and the system; an alternative step such as 4-A replaces the system response with the same number.

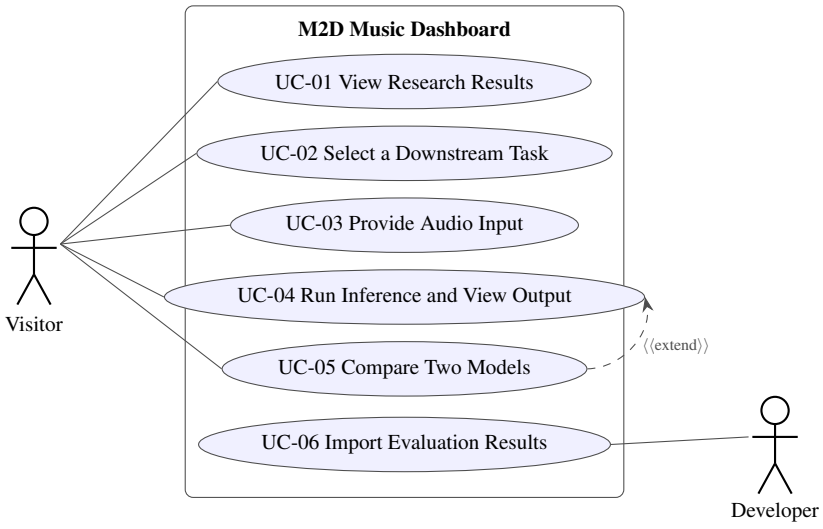


Figure 4.1: Use-case diagram of the dashboard. The Visitor takes part in UC-01 to UC-05 and the Developer in UC-06; UC-05 extends UC-04.

Table 4.2: Use case UC-01: View Research Results.

Name	View Research Results		
Actors	Visitor		
Summary	The visitor opens the dashboard and reviews the results of the further pre-training study: the music scores, the per-task scores and a chart, filtered by model group.		
Pre-Conditions	The server is running. The developer has imported the evaluation results (UC-06).		
Post-Conditions	The results of the selected model group are displayed. The database is not changed.		
Special Requirements	The scores shall be identical to those reported in the results chapter of this report.		
Basic Flow			
Actor Action		**System Response**	
1	The visitor opens the dashboard address in a browser.	2	The system loads the results from the database and displays the Results Overview with S_{mus}, the per-task table and the bar chart.
3	The visitor selects a model group in the filter.	4	The system updates the table and the chart to show only the models of that group.
Alternative Flow			
1	The visitor opens the dashboard before any result has been imported.	2-A	The system displays the message “No results have been imported yet” and keeps the Demo and About tabs available.

Table 4.3: Use case UC-02: Select a Downstream Task.

Name	Select a Downstream Task		
Actors	Visitor		
Summary	The visitor chooses one of the four demonstration tasks, and the system shows the labels and the input expected by that task.		
Pre-Conditions	The server is running and probes of at least one model exist for the task.		
Post-Conditions	The task is selected and the audio input panel is enabled for it.		
Special Requirements	For the instrument and pitch tasks the system notes that a single isolated note is expected, as in NSynth.		
Basic Flow			
Actor Action		**System Response**	
1	The visitor opens the Demo tab.	2	The system displays four task cards with the dataset, the number of labels and the metric of each task.
3	The visitor clicks a task card.	4	The system highlights the card, shows the expected input and enables the audio input panel and a model list that contains only the models with a probe for the task.
Alternative Flow			
3	The visitor clicks a task for which no probe has been registered.	4-A	The system shows the card as unavailable with the message “This task is not available yet”.

Table 4.4: Use case UC-03: Provide Audio Input.

Name	Provide Audio Input		
Actors	Visitor		
Summary	The visitor uploads an audio file or selects a bundled sample clip.		
Pre-Conditions	A task has been selected (UC-02).		
Post-Conditions	A valid clip is held by the server for the current request, and a player and a log-mel preview of it are displayed.		
Special Requirements	Accepted formats are WAV and MP3, with at most 10 MB per file; only the first 6 s of a clip are used. Uploaded files are deleted after the prediction.		
Basic Flow			
Actor Action		**System Response**	
1	The visitor drags an audio file into the upload area, or clicks Browse and selects a file.	2	The system checks the type and size, decodes the audio, and displays its duration, a player and a log-mel preview.
3	Alternatively, the visitor selects a sample clip from the list.	4	The system loads the clip and displays its source dataset, license and source label, a player and a log-mel preview.
Alternative Flow			
1	The visitor uploads a file in an unsupported format.	2-A	The system displays “Unsupported file format. Please use WAV or MP3.”
1	The visitor uploads a file larger than 10 MB.	2-B	The system displays “File too large (maximum 10 MB).”
1	The visitor uploads a clip longer than 6 s.	2-C	The system accepts the file and displays “Only the first 6 s will be analyzed.”
1	The visitor uploads a corrupted file.	2-D	The system displays “The file could not be read.”

Table 4.5: Use case UC-04: Run Inference and View Output.

Name	Run Inference and View Output		
Actors	Visitor		
Summary	The visitor runs the selected model on the clip and sees the prediction.		
Pre-Conditions	A task has been selected (UC-02), a valid clip is available (UC-03) and a model has been selected.		
Post-Conditions	The prediction is displayed, an anonymous record is added to the inference log and an uploaded file is deleted.		
Special Requirements	The preprocessing and embedding are those of the evaluation pipeline. The response time follows the targets of Section 4.4.		
Basic Flow			
Actor Action		**System Response**	
1	The visitor selects a model (the further pre-trained M2D by default) and clicks Run.	2	The system computes the log-mel spectrogram and the clip embedding, and applies the linear probe of the model and task.
3	The visitor reads the result.	4	The system displays the task-specific output, the inference time, the device and, for a sample clip, the source label.
Alternative Flow			
1	The visitor clicks Run while another request is being processed.	2-A	The system queues the request, shows its position and starts it when the model is free.
1	The visitor clicks Run on a host without a usable GPU.	2-B	The system runs the model on the CPU and notes that the response is slower.
1	The visitor clicks Run and the inference fails.	2-C	The system displays “The prediction failed. Please try again.” and logs the failure.

Table 4.6: Use case UC-05: Compare Two Models on a Clip.

Name	Compare Two Models on a Clip		
Actors	Visitor		
Summary	The visitor runs both the original M2D and the further pre-trained M2D on the same clip and sees their outputs side by side.		
Pre-Conditions	As in UC-04, and probes of both models exist for the selected task.		
Post-Conditions	Both predictions are displayed side by side and two records are added to the inference log.		
Special Requirements	Both models receive the same decoded clip.		
Basic Flow			
Actor Action		**System Response**	
1	The visitor ticks Compare with.	2	The system adds the other M2D model to the request.
3	The visitor clicks Run.	4	The system runs both models one after the other and displays the outputs in two columns, highlighting the labels predicted by only one model.
Alternative Flow			
3	The visitor clicks Compare with the other model on an output page.	4-A	The system reuses the clip, runs only the other model and opens the comparison view.

Table 4.7: Use case UC-06: Import Evaluation Results.

Name	Import Evaluation Results		
Actors	Developer		
Summary	The developer runs a command-line script that reads the result files of the evaluation pipeline and registers models, probes, results and sample clips in the database.		
Pre-Conditions	The evaluation has finished and its result files exist in the agreed CSV or JSON format. The developer has access to the server.		
Post-Conditions	The database contains the new rows, and the Results Overview shows them after a refresh.		
Special Requirements	The import is transactional: either all rows of a file are written or none.		
Basic Flow			
Actor Action		**System Response**	
1	The developer runs the import script with the path of a result file.	2	The script checks every record (known model and task codes, the metric of the task, a score between 0 and 100, at least one seed) and writes all rows in one transaction.
3	The developer opens the dashboard.	4	The system displays the updated results.
Alternative Flow			
1	The file contains an unknown model or task code or a missing field.	2-A	The script rejects the whole file, reports the line and the field, and writes nothing.
1	Results for the same model, task and split already exist.	2-B	The script stops, unless the replace option is given, in which case the old rows are replaced.

4.7 Hardware and Software Requirements

The dashboard is small and reuses the environment of the research, so its requirements are modest. The following hardware and software are needed to develop and deploy it.

4.7.1 Hardware Requirements

The following are the hard requirements for the host and the visitors' computers. The host is the machine that runs the server and holds the models, probes and database; the visitors are the computers that open the dashboard in a web browser.

- A development and hosting workstation; the research workstation with one NVIDIA RTX A4000 GPU (16 GB) will be used. A GPU is recommended for fast predictions but is not required.
- At least 16 GB of system memory and about 10 GB of free disk space for the checkpoints, probes, sample clips and database.
- A desktop or laptop computer with a web browser for each visitor, with speakers or headphones to play clips.
- A local network connection between the host and the visitors' computers, or the host itself for a single-screen demonstration.

4.7.2 Software Requirements

The following software is needed on the host to run the server and on the visitors' computers to open the dashboard in a web browser.

- Linux operating system with the NVIDIA driver and CUDA runtime on the host.
- Python 3.10 or above, the language of the backend and of the research code.
- PyTorch and torchaudio for model inference, with the M2D code base and timm [17] to build the encoder.
- nnAudio [21] for the log-mel front-end, as in the evaluation pipeline.
- soundfile and FFmpeg to decode the accepted audio formats.
- FastAPI and the Uvicorn server for the JSON API and for serving the frontend files.
- SQLite with the SQLAlchemy library for the database.
- HTML5, CSS3 and JavaScript for the frontend, with the Chart.js library for charts.

- Git and GitHub for version control, and Visual Studio Code as editor.
- A current web browser such as Google Chrome, Mozilla Firefox or Microsoft Edge.

4.8 Graphical User Interface

This section presents the planned screens of the dashboard as wireframes; screenshots of the implemented dashboard will replace them in the chapter on implementation. All screens share a top bar with the dashboard name and three tabs, Results, Demo and About, so that every view can be reached from every screen. Values shown in the wireframes are placeholders. Figure 4.2 shows the navigation flow, and Figures 4.3 to 4.7 show the screens with the use cases that each one covers. The visitor starts on the Results Overview. The Demo tab leads to the task and input screen; clicking Run leads to the output screen, or to the comparison view when a second model is selected. From the output screen the visitor can go back to change the input or task, or open the comparison view. The Results Overview (Figure 4.3) has a model-group filter; three summary cards with the music score of the original M2D, the music score of the further pre-trained M2D and the change between them; a bar chart of the mean music score per model, in which the two M2D models are highlighted and the reference models are shown for context; and a table of per-task scores with the change for each task.

The Demo screen (Figure 4.4) is organized in three steps: the visitor selects one of four task cards (genre, tagging, instrument and pitch); uploads a file through a drag-and-drop area or picks a sample clip, which can be played and is shown as a log-mel spectrogram; and selects a model, with a Compare with check box that adds the original M2D, before clicking Run. A disclaimer is shown at the bottom. The output screen (Figure 4.5) lists the input, the model, the device and the inference time next to the scores: the five most probable genres, the tags above the threshold and the ten highest tag scores, the three most probable instrument families, or the three most probable pitches with MIDI number, note name and frequency. Buttons change the input or task or run the original M2D on the same clip. The comparison view (Figure 4.6) shows the outputs of the two models side by side; for tagging, a dashed line marks the decision threshold of 0.5, and a tag predicted by only one model is shown in bold. The About page (Figure 4.7) describes the project, its method, the models, the datasets and the sample-clip licenses, and a highlighted box states that single-clip predictions are a demonstration and that the benchmark scores are those on the Results tab.

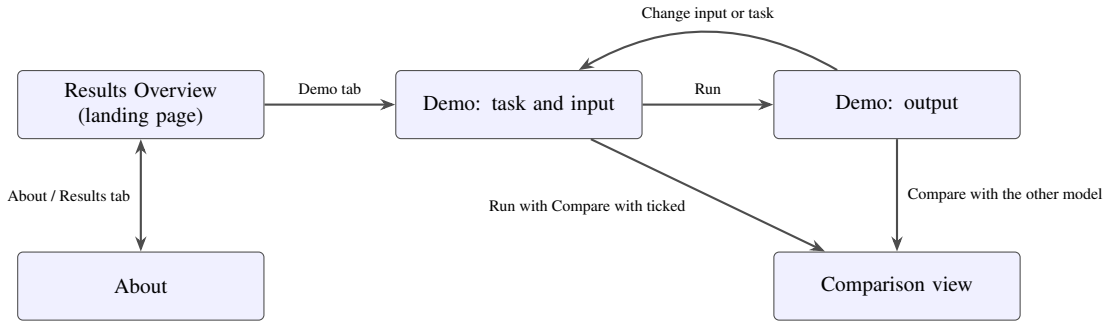


Figure 4.2: Navigation flow of the dashboard. Arrows show the transitions between screens, and the About page is reachable from every screen through the tabs.

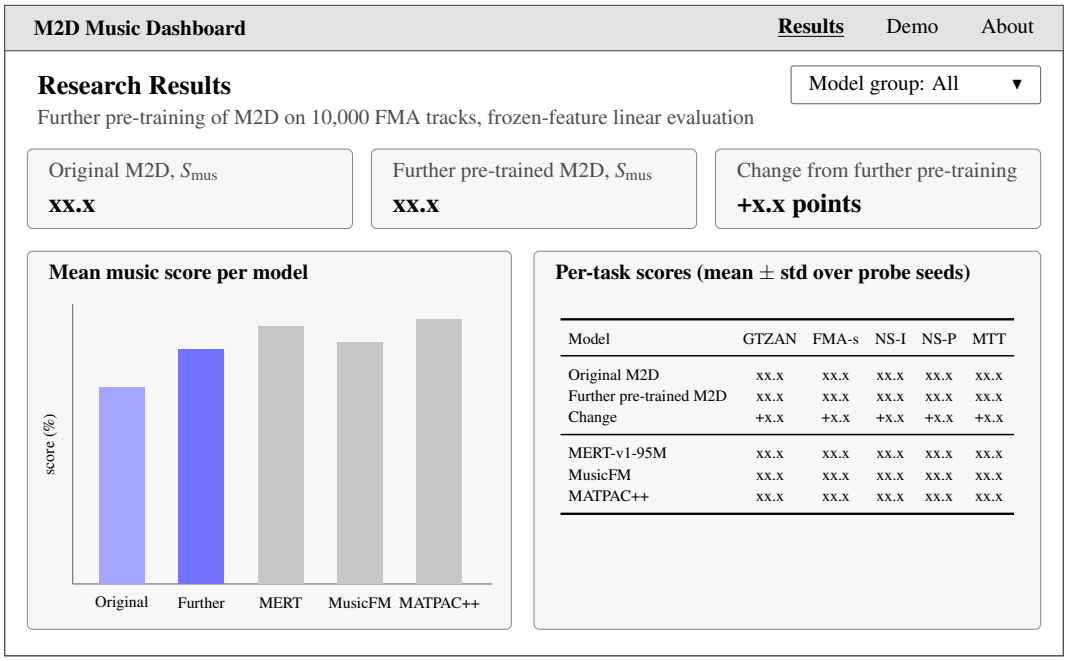


Figure 4.3: Wireframe of the Results Overview (UC-01). The page shows summary cards, a bar chart of the music score per model and a table of per-task scores; values are placeholders.

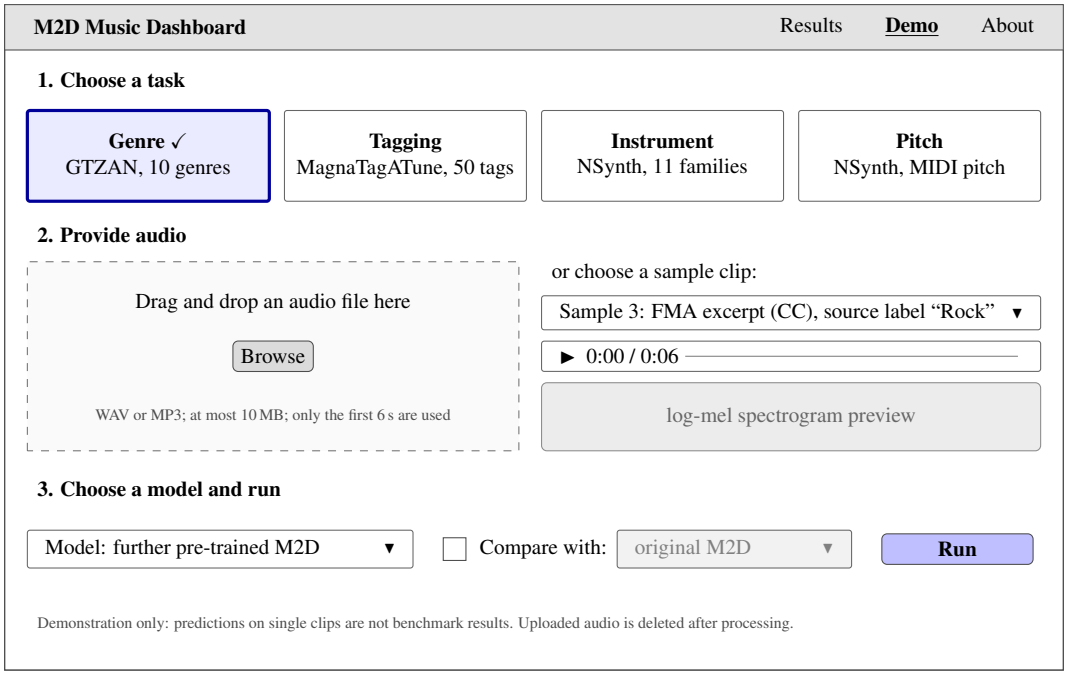


Figure 4.4: Wireframe of the Demo screen for task and input (UC-02, UC-03). The visitor picks a task, uploads or selects a clip, and chooses one or two models before running.

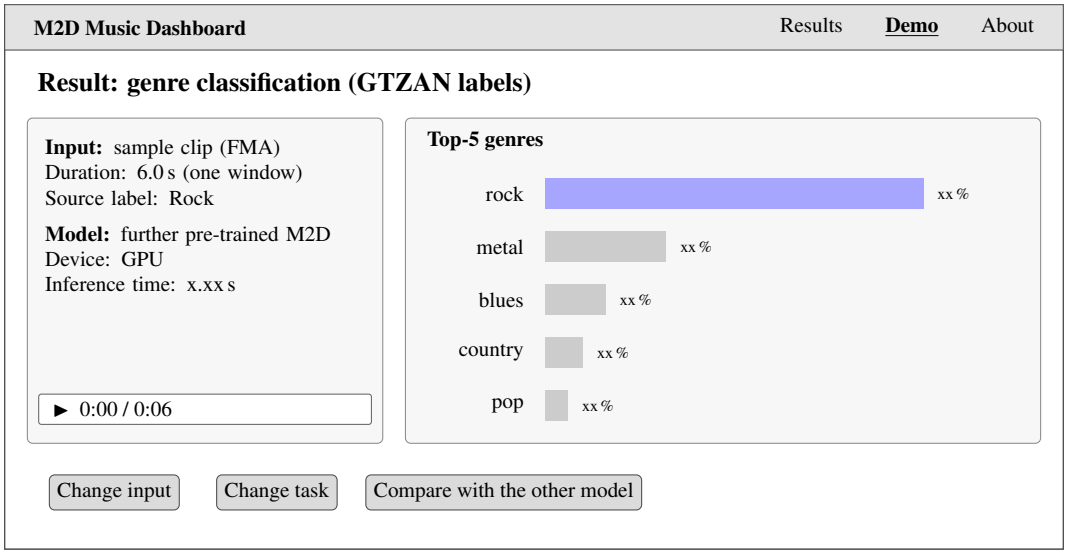


Figure 4.5: Wireframe of the Demo output screen (UC-04), shown for the genre task. Details of the input and model are on the left and the five most probable genres on the right.

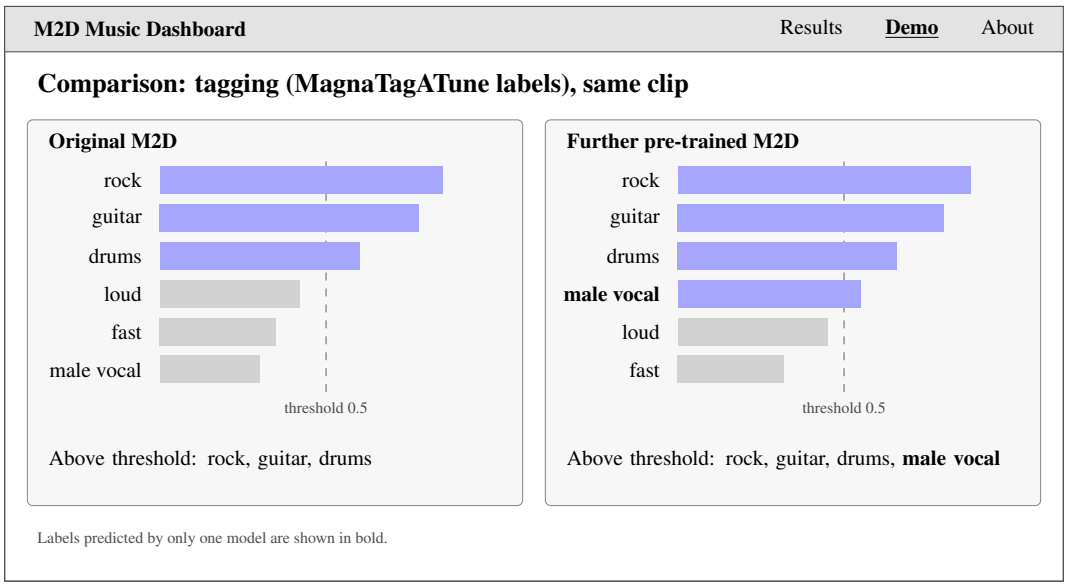


Figure 4.6: Wireframe of the comparison view (UC-05), shown for the tagging task. The outputs of the original and the further pre-trained M2D are shown side by side for one clip.

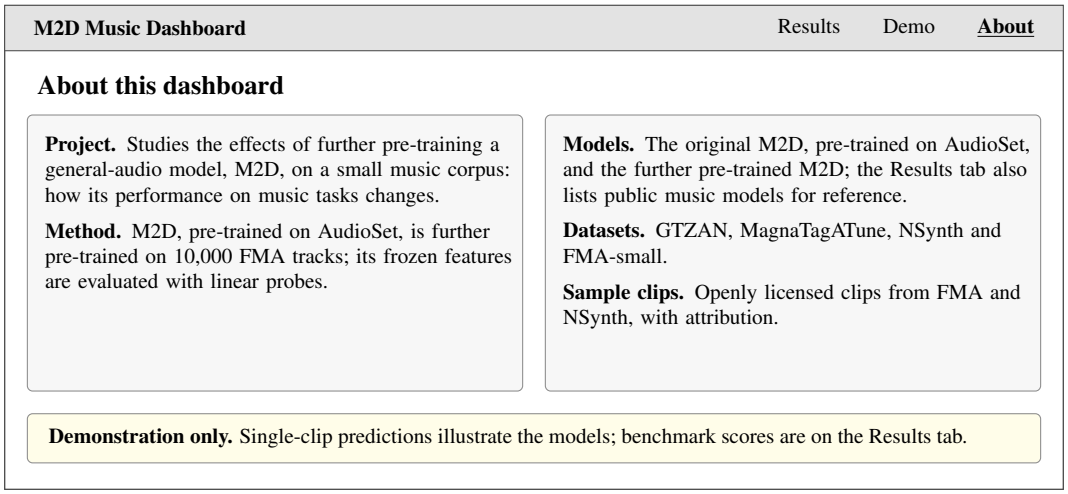


Figure 4.7: Wireframe of the About page (FR-18). It describes the project, method, models, datasets and licenses, and notes that single-clip predictions are only a demonstration.

4.9 Database Design

The dashboard needs a small database for the research data it displays (models, tasks, labels, probes and results) and the demonstration (sample clips and an anonymous inference log); there is no user data, as there are no accounts. SQLite is used because the dashboard runs on one host with small, mostly read data, and is accessed through SQLAlchemy so that a client–server database could replace it later. Reference models are stored only with their results, and FMA-small appears only on the Results tab.

4.9.1 ER Diagram

Figure 4.8 shows the entities and their relationships. `Model` and `Task` are the central entities. A probe, an evaluation result and an inference-log record each belong to one model and one task, so each of the three entities is in a one-to-many relationship with both. A task has many labels and many sample clips. Audio files and probe weights are stored on disk, and the database keeps only their paths.

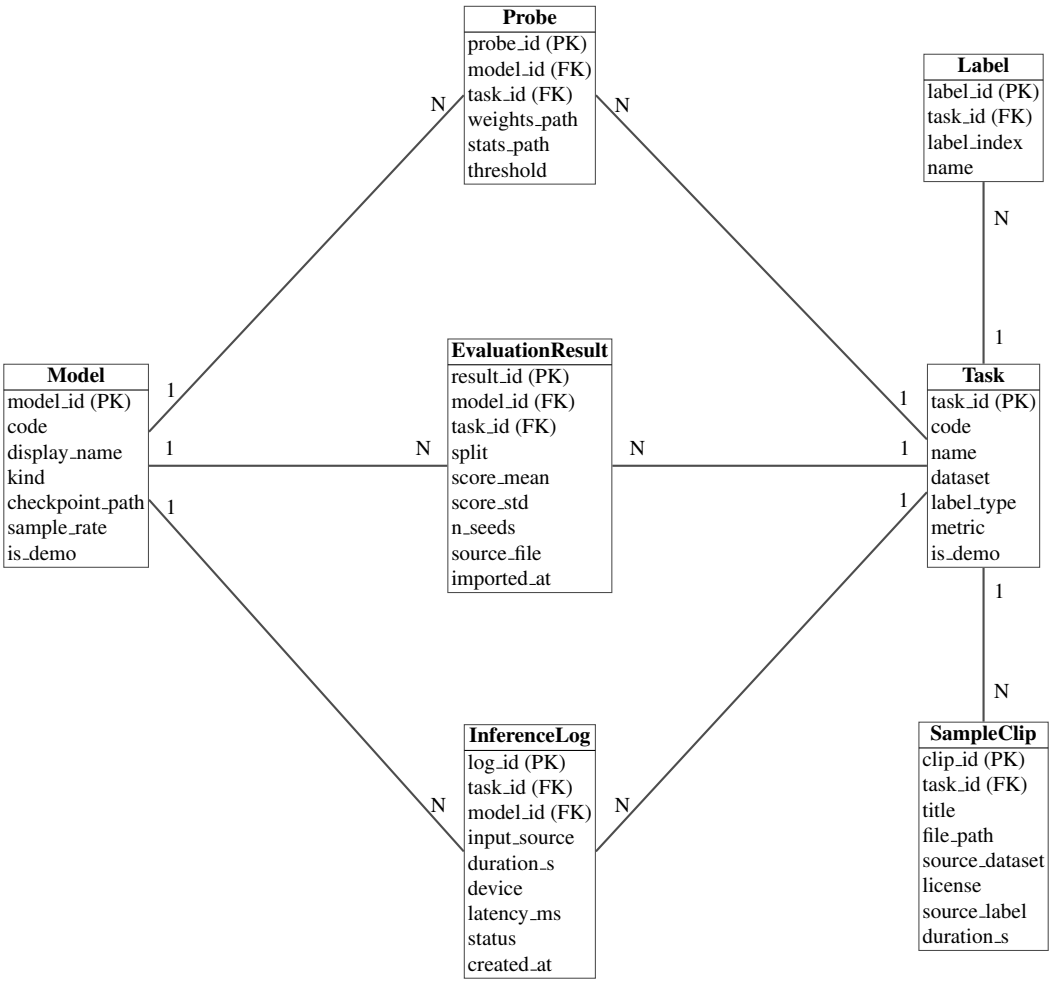


Figure 4.8: ER diagram of the dashboard database. Primary and foreign keys are marked PK and FK, and 1 and N give the cardinality of each relationship.

4.9.2 Data Dictionary

Table 4.8 lists the attributes of every entity. SQLite has no separate Boolean and date types; Boolean values are stored as integers 0 and 1, and time stamps as ISO 8601 text.

Table 4.8: Data dictionary of the dashboard database. For each entity the table gives the attributes, their data types, whether they may be empty, and a description.

Entity	Attribute	Data Type	Nullable	Description
Model	model_id	Integer	No	Primary key.
	code	Text	No	Unique short code, for example m2d-original, m2d-music or mert-v1-95m.
	display_name	Text	No	Name shown in the interface.
	kind	Text	No	One of original, further-pretrained or reference.
	checkpoint_path	Text	Yes	Path of the checkpoint on disk; empty for reference models, which are shown only on the Results tab.
	sample_rate	Integer	No	Input sampling rate in Hz (16,000 for M2D).
	is_demo	Boolean	No	Whether the model can be selected on the Demo tab.
Task	task_id	Integer	No	Primary key.
	code	Text	No	Unique short code, for example genre, tagging, instrument, pitch, fma-small.
	name	Text	No	Name shown in the interface.
	dataset	Text	No	Evaluation dataset, for example GTZAN or NSynth.
	label_type	Text	No	single for single-label or multi for multi-label tasks.
	metric	Text	No	accuracy or mAP.
	is_demo	Boolean	No	Whether the task is offered on the Demo tab.
Label	label_id	Integer	No	Primary key.
	task_id	Integer	No	Foreign key to Task.
	label_index	Integer	No	Output index of the label in the probe.
	name	Text	No	Label name, for example rock, guitar, brass or a MIDI pitch.
Probe	probe_id	Integer	No	Primary key.
	model_id	Integer	No	Foreign key to Model.
	task_id	Integer	No	Foreign key to Task; the pair (model_id, task_id) is unique.
	weights_path	Text	No	Path of the linear-layer weights.
	stats_path	Text	No	Path of the mean and standard deviation used to standardize embeddings.
	threshold	Real	Yes	Decision threshold for multi-label tasks (0.5 by default); empty for single-label tasks.
EvaluationResult	result_id	Integer	No	Primary key.
	model_id	Integer	No	Foreign key to Model.

Entity	Attribute	Data Type	Nullable	Description
	task_id	Integer	No	Foreign key to Task.
	split	Text	No	validation or test; (model_id, task_id, split) is unique.
	score_mean	Real	No	Mean score over probe seeds, in percent.
	score_std	Real	Yes	Standard deviation over probe seeds.
	n_seeds	Integer	No	Number of probe seeds.
	source_file	Text	No	Result file from which the row was imported.
	imported_at	Date-time	No	Time of import.
SampleClip	clip_id	Integer	No	Primary key.
	task_id	Integer	No	Foreign key to Task for which the clip is offered.
	title	Text	No	Name shown in the sample list.
	file_path	Text	No	Path of the audio file on disk.
	source_dataset	Text	No	Dataset from which the clip comes, for example FMA or NSynth.
	license	Text	No	License of the clip and attribution text.
	source_label	Text	Yes	Label given by the source dataset, shown for reference.
	duration_s	Real	No	Duration in seconds.
InferenceLog	log_id	Integer	No	Primary key.
	task_id	Integer	No	Foreign key to Task.
	model_id	Integer	No	Foreign key to Model.
	input_source	Text	No	upload or sample.
	duration_s	Real	Yes	Duration of the analyzed audio; empty if decoding failed.
	device	Text	No	gpu or cpu.
	latency_ms	Integer	Yes	Processing time in milliseconds; empty if the request failed.
	status	Text	No	ok or an error code.
	created_at	Date-time	No	Time of the request.

4.10 Risk Analysis

The dashboard is a small part of a research project with a fixed three-month schedule and a single GPU, so its risks are mostly about timing, faithfulness and licensing rather than scale. The main risks are described below and summarized with their mitigation in Table 4.9.

4.10.1 Technical Risks

The dashboard depends on the outputs of the research: if the further pre-trained M2D or its probes are late, there is nothing to demonstrate for it. The dashboard is therefore developed first with the original M2D, whose probes are available early, and the further pre-trained M2D is added by import. A second risk is a silent mismatch between the preprocessing of the dashboard and that of the evaluation, which would make the demonstration show a model that behaves differently from the one evaluated;

it is reduced by importing the evaluation code instead of copying it and by a check that the dashboard reproduces the cached embeddings of a few test clips. Finally, the GPU may be busy with training or unavailable at a presentation, in which case the CPU path is used, and some uploaded files may fail to decode, which is handled by the input checks.

4.10.2 Schedule Risks

The research has priority over the dashboard in the three-month schedule. The dashboard is kept minimal, with no accounts and a single page, and it is built in small steps (results view, then one task, then the other tasks, then comparison) so that a usable version exists at every stage.

4.10.3 Legal and Licensing Risks

Bundled sample clips are redistributed to visitors, so they must have suitable licenses. Sample clips are taken only from openly licensed sources such as FMA [13] and NSynth [45] and shown with their license and attribution; GTZAN clips, which are excerpts of commercial recordings, are not bundled. Reference models appear only through their evaluation scores, so their weights, some of which are licensed for non-commercial use only [11], are neither used by nor distributed with the dashboard.

4.10.4 Misinterpretation Risks

Visitors may read a single prediction as evidence that one model is better than another. The dashboard states on the Demo and About pages that single-clip predictions are illustrations, and the Results tab shows only the scores of the full evaluation, identical to those of the report.

Table 4.9: Summary of risks of the dashboard with their likelihood, impact and mitigation.

Risk	Likelihood	Impact	Mitigation
Checkpoint or probes of the further pre-trained M2D are late	Medium	High	Develop with the original M2D first; add the further pre-trained M2D by import
Preprocessing differs from evaluation	Medium	High	Import the evaluation code; check against cached embeddings
GPU unavailable at a presentation	Low	Medium	CPU fallback; models loaded at start-up
Uploaded files fail to decode	Medium	Low	Type, size and decoding checks with clear messages
Dashboard work delays the research	Medium	Medium	Minimal scope; incremental development
Unsuitable licenses of sample clips	Low	High	Only openly licensed clips with attribution; no GTZAN clips
Single predictions taken as results	Medium	Medium	Disclaimers; Results tab shows only evaluation scores

4.11 Conclusion

This chapter specified the M2D Music Dashboard, the small web application that accompanies the study. It presents the results of the further pre-training experiments and demonstrates four MARBLE tasks, namely genre, tagging, instrument and pitch, on clips that a visitor uploads or selects, with a side-by-side comparison of the original and the further pre-trained model. The requirements favor simplicity and faithfulness: there are no accounts, a single page with three tabs, a FastAPI backend that reuses the evaluation code and probes, a plain HTML and JavaScript frontend, and a small SQLite database whose results are imported unchanged from the evaluation. The use cases, wireframes, database design and risk analysis provide the basis for the design in the next chapters, while the research method that produces the models and results is described in Chapter 5.

Chapter 5 Proposed Approach and Methodology

This chapter describes how the study is carried out. The AudioSet-pretrained M2D model is further pre-trained on 10,000 tracks of the Free Music Archive (FMA) and compared, with frozen features, against its unmodified version and against public music-specific models. Figure 5.1 shows the four parts of the approach: the acquisition and preprocessing of the data (Sections 5.1 and 5.2), the further pre-training framework, called M2D-X-Music in this report (Section 5.3), the experimental arms (Section 5.4) and the evaluation protocol (Section 5.5).

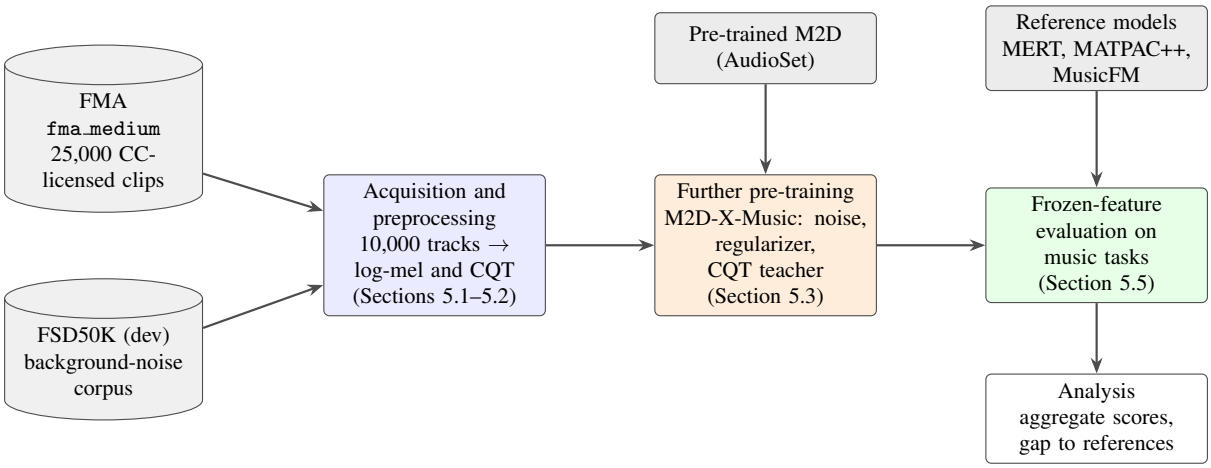


Figure 5.1: Overview of the proposed methodology. FMA and noise data are prepared, M2D is further pre-trained with M2D-X-Music, and the checkpoints are compared with reference models on music tasks.

5.1 Dataset Acquisition

This section describes the source of the pre-training audio and how the 10,000-track subset is drawn. Throughout this report a sample is one 30-s FMA clip, that is, one track, so 10,000 samples correspond to about 83.3 h of music, or about 1.5 % of the 5,569 h of AudioSet on which M2D was trained.

5.1.1 Choice of Corpus

The pre-training corpus must be usable for research, provide full-quality audio, identify artists and provide predefined splits so that leakage into the evaluation can be controlled, and be small enough to process on one workstation. The Free Music Archive (FMA) [13] meets these requirements: its audio is released under Creative Commons licenses chosen by the artists, the tracks are MP3 files, most of them stereo at 44.1 kHz and 320 kbit/s, and the metadata include artist identifiers and an official, artist-filtered split.

5.1.2 Archives and Metadata

FMA is distributed as one metadata archive (`fma_metadata.zip`) and four audio archives that are nested subsets of one another [13, 40]. The metadata table `tracks.csv` describes all 106,574 tracks, with artist, album, genre, license, and the columns that give the subset and the official training, validation and test split of each track. In the medium, small and large subsets the audio of a track is a 30-s clip cut from the middle of the full track, or the whole track if it is shorter. The medium subset contains 25,000 clips with a single top-level genre each, selected according to the completeness of their metadata and their popularity in the hope of selecting tracks of higher quality; the small subset is the balanced selection of 1,000 clips from each of the eight most popular genres of the medium subset [13]. The splits are 80 %/10 %/10 % for training, validation and test, stratified by genre, with an artist filter so that every artist appears in only one split, and a track keeps the same split in every subset [13].

Table 5.1: FMA audio archives and their use in this study. Sizes are those listed by the dataset maintainers [40]; clip counts and genre counts are from the dataset paper [13].

Archive	Clips	Genres	Size (GiB)	Use in this study
`fma_small`	8,000	8 (balanced)	7.2	Audio contained in the medium archive; official test split used for in-domain genre evaluation
`fma_medium`	25,000	16 (unbalanced)	22	Source of the pre-training pool (training split only) and of the `fma_small` evaluation clips
`fma_large`	106,574	161	93	Not used (larger download, no additional split advantage)
`fma_full`	106,574	161	879	Not used (untrimmed tracks)

The study downloads the metadata archive and the medium audio archive only, verifies the SHA-1 checksums published with the dataset, and unpacks the files. Because the small subset is contained in the medium subset, the single audio archive also supplies the clips for the in-domain evaluation.

5.1.3 Eligibility Rules and Subset Selection

A track enters the candidate list only if it satisfies all of the following rules.

1. It belongs to the small or medium subset and to the training split. Since the official splits are artist-filtered, no artist from the validation or test split contributes to pre-training, which prevents in-domain leakage into the FMA evaluation task. About 80 % of the 25,000 clips, that is, about 20,000 tracks, are expected to qualify; the exact count is recorded when the metadata are read.
2. Its license field is present in the metadata.
3. It passes the audio audit described in Section 5.2.
4. At most $k = 5$ tracks of the same artist are admitted. This limits the influence of prolific artists in

a corpus with a long-tailed artist distribution.

No genre label is used for selection, so the subset keeps the genre distribution of the medium subset, in which experimental music is under-represented [13]; the genre histogram of the chosen subset is reported so that this property is visible. Pre-training itself uses no labels at all.

Algorithm 1 fixes the selection. A seeded random permutation of the eligible tracks defines the order in which candidates are examined. The first 10,000 accepted tracks form the main subset, and shorter prefixes of the same list form the nested subsets with 1,000, 2,500 and 5,000 tracks, plus a 20,000-track extension if time allows. Using prefixes of one list means that all arms and all sizes draw on the same data, so a difference between two results cannot come from a different selection of tracks. Only the ordered list of track identifiers and the seed need to be stored to reproduce the subset.

Data: metadata table T; seed s; artist cap k; sizes $n_1 < n_2 < \cdots < n_J$
Result: ordered list L and nested subsets $S_j = L[1..n_j]$; rejection log
$E \leftarrow \{t \in T : t.\text{subset} \in \{\text{small}, \text{medium}\},\ t.\text{split} = \text{training},\ t.\text{license} \neq \emptyset\}$;
$\pi \leftarrow$ random permutation of E generated with seed s;
$L \leftarrow$ empty list; count$[a] \leftarrow 0$ for every artist a;
foreach *track* t *in* π*, in order* **do**
  if count$[t.\text{artist}] < k$ ***and*** Audit(t) *passes* **then**
    append t to L;
    count$[t.\text{artist}] \leftarrow$ count$[t.\text{artist}] + 1$;
  end
  if $|L| = n_J$ **then** leave the loop;
end
return L, the subsets S_j, and the log of rejected tracks with their reasons;

Algorithm 1: Seeded, artist-capped selection of nested pre-training subsets

5.1.4 Background-Noise Corpus

The denoising component of M2D-X needs a corpus of background sounds. The official recipe uses the FSD50K dataset [15, 4], which contains 51,197 Freesound clips (108.3 h) under Creative Commons licenses, labeled with 200 classes of the AudioSet ontology. Only the development portion is used as noise, and the evaluation portion is left untouched. Because the ontology includes musical instruments, an optional ablation restricts the pool to clips that carry no music-related label, to test whether purely non-musical noise is a better distractor for music (Section 5.4).

5.2 Data Preprocessing Pipeline

Figure 5.2 shows the pipeline. Its guiding rule is that the audio must reach the model through exactly the same front-end as in the original pre-training, because a pre-trained encoder that is fed differently computed spectrograms suffers a distribution shift that would be confounded with the effect under study.

The top row of the figure operates on metadata and audit results and fixes the ordered list of tracks. The bottom row converts the selected files with the same front-end that was used to pre-train M2D. The noise corpus passes through the log-mel stage only.

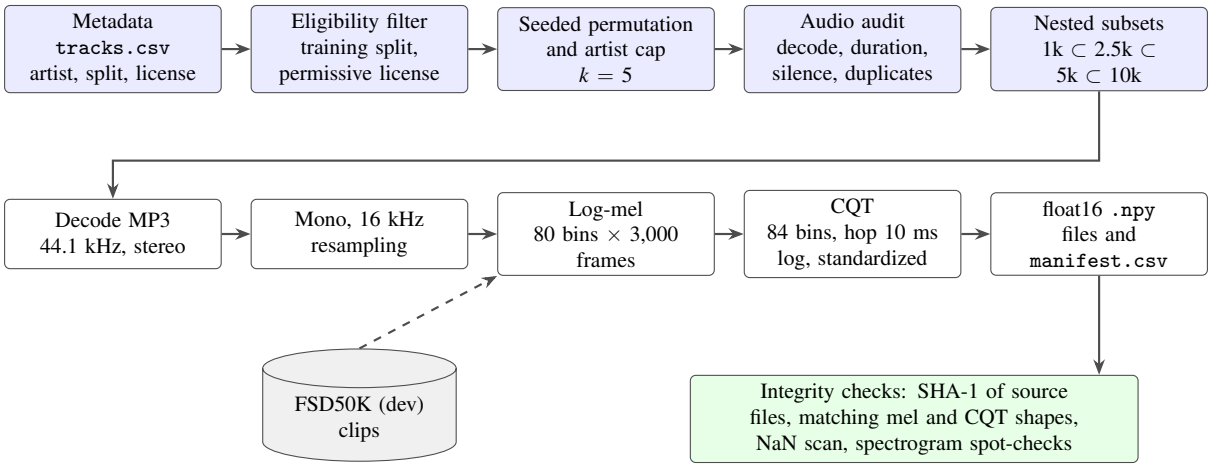


Figure 5.2: Data acquisition and preprocessing pipeline. The top row selects the tracks and the bottom row converts them; the noise corpus passes through the log-mel stage only (dashed arrow).

5.2.1 Audit and Decoding

Each candidate MP3 file is decoded before it is accepted. The FMA maintainers document corrupted or truncated tracks, tracks shorter than 30 s, and duplicates [40], so the audit applies four rejection rules.

1. The decoder reports an error or a truncated stream.
2. The decoded signal is shorter than 10 s, so that several different 6.08-s windows cannot be cut from it.
3. More than half of the frames have a power more than 60 dB below that of the loudest frame of the clip.
4. The track has the same normalized pair of artist identifier and title as an accepted track, or the same hash of the decoded 16-kHz waveform. The track with the lowest identifier is kept.

Every rejection is logged with its reason, and the next candidate in the permutation takes the place of a rejected track (Algorithm 1).

5.2.2 Front-End

The conversion script distributed with the M2D code (`wav_to_lms.py`) [17] is used without modification. It loads each file as a mono signal resampled to 16 kHz and computes a log-mel spectrogram with nnAudio [21]. With the mel filter bank H and the short-time Fourier transform X of the waveform, each

spectrogram entry is

$$x_{f,t} = \log\Big(\sum_k H_{f,k}\,|X_{k,t}|^2 + \varepsilon\Big), \qquad f = 1,\ldots,80, \tag{5.1}$$

where ε is a small constant that avoids the logarithm of zero. Table 5.2 lists all parameters. Spectrograms are stored in half precision, which halves the disk space (about 4.8 GB for 10,000 clips of 3,000 frames) at a rounding error far below the standard deviation of the data, and they are cast to single precision when loaded.

Table 5.2: Front-end and target parameters. Values of the log-mel front-end and of the input geometry follow the released M2D model [17, 4]; the CQT settings follow the official MERT implementation [23], with the hop size changed to match the log-mel frame grid.

Item	Value
Audio decoding	Channel mean to mono, resampled to 16 kHz
STFT	Window and FFT size 400 samples (25 ms), hop 160 samples (10 ms)
Mel bands	80 bands between 50 Hz and 8,000 Hz
Input window	608 frames (≈6.08 s); patches of 16×16; $N_F = 5$, $N_T = 38$, $N = 190$ patches
Normalization	Mean $\mu_{\mathrm{AS}} = -7.1$ and standard deviation $\sigma_{\mathrm{AS}} = 4.2$ of the AudioSet pre-training data
CQT target	Magnitude, 84 bins at 12 per octave from 32.7 Hz (C1) to about 3.95 kHz, hop 160 samples; logarithm, standardized per bin
Target token rate	Average over $P = 16$ frames (160 ms) to match one patch step

5.2.3 Cropping and Normalization

At every visit, a random window of 608 frames is cut from the stored 30-s spectrogram, as in the official loader. A 30-s track contains about five non-overlapping windows, and random start positions give many more overlapping ones, so the 10,000 tracks supply a large number of distinct inputs even though the track count is small. The window is standardized as

$$\bar{x} = \frac{x - \mu_{\mathrm{AS}}}{\sigma_{\mathrm{AS}}}, \tag{5.2}$$

with the AudioSet statistics of Table 5.2, so that the pre-trained encoder initially sees inputs of the same scale as during its original pre-training. Unlike the official M2D-X code, which recomputes these statistics from the application data, fixed statistics remove one source of change at the start of training; the statistics of the selected FMA subset are reported, and training with them is an optional run.

5.2.4 Musical-Teacher Targets

The musical teacher uses the constant-Q transform [46], whose center frequencies are spaced logarithmically,

$$f_k = f_{\min} 2^{(k-1)/12}, \qquad k = 1, \ldots, 84, \quad f_{\min} = 32.7 \text{ Hz}, \tag{5.3}$$

so that every semitone has its own bin and the top bin lies at about 3.95 kHz, below the Nyquist frequency of 8 kHz. Let $C_{k,t}$ be the CQT magnitude computed with nnAudio on the same 16-kHz mono waveform and with the same hop of 160 samples as the log-mel spectrogram. The target is the log magnitude standardized per bin with the mean m_k and standard deviation s_k over the selected subset, followed by average pooling over the $P = 16$ frames of each token time step τ:

$$\tilde{c}_{k,t} = \frac{\log(C_{k,t} + \varepsilon) - m_k}{s_k}, \qquad \bar{c}_{k,\tau} = \frac{1}{P} \sum_{t=(\tau-1)P+1}^{\tau P} \tilde{c}_{k,t}. \tag{5.4}$$

Unlike MERT, which regresses the linear magnitude at its own frame rate [23], the logarithm and the per-bin standardization keep the target on the scale of the log-mel input, and the pooling matches the 160-ms time step of an M2D token. Because the two transforms use the same hop and centered frames, the log-mel and CQT arrays of a clip have the same number of frames; if they differ by a frame because of padding, both are trimmed to the shorter, and one random start index is used to crop both.

5.2.5 Storage, Manifest and Integrity Checks

For each accepted track the pipeline writes one half-precision array for the log-mel spectrogram (about 4.8 GB for 10,000 clips) and one for the CQT (about 5.0 GB), and a row in a manifest file with the track identifier, artist identifier, file paths, frame counts, SHA-1 of the source file, audit outcome and split. The noise corpus is converted to log-mel arrays only. The following checks are run and their outcome is saved: the SHA-1 of every source file matches the published checksum; log-mel and CQT frame counts agree; no array contains NaN or infinite values; value ranges lie within a plausible interval of the normalized scale; and a gallery of 20 randomly chosen spectrograms is inspected by eye against the audio. The conversion is resumable and driven by one configuration file, so an interrupted run does not repeat finished tracks.

5.3 Further Pre-training Framework

This section defines M2D-X-Music, the objective and the training procedure applied to the AudioSet-pretrained M2D. Figure 5.3 shows the data flow.

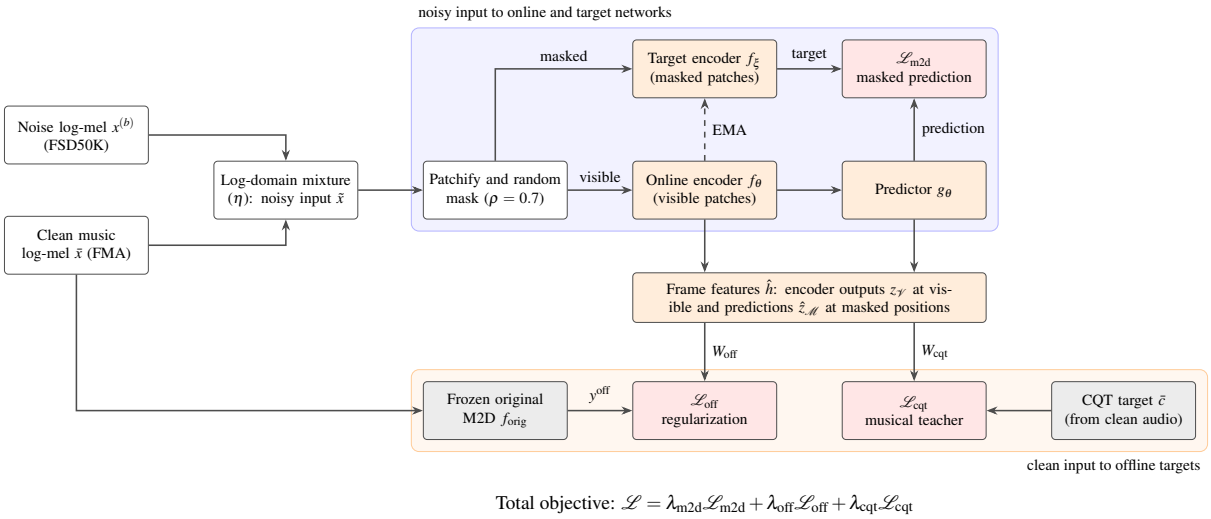


Figure 5.3: Architecture of the further pre-training framework. The blue region processes the noisy input and the orange region gives the clean targets; only the online encoder f_θ is kept.

5.3.1 Initialization

Training starts from the released checkpoint M2D/0.7 (80×608 input, 16×16 patches, masking ratio 0.7) [17]. The online encoder f_θ, the predictor g_θ and the EMA target encoder f_ξ are all initialized from this checkpoint, and a frozen copy f_{orig} of the original encoder serves as offline teacher. Two linear mappers, W_{off} and W_{cqt}, are newly initialized. The online and target networks see the noisy input, whereas the offline branch sees the clean input and its precomputed targets. After pre-training only the online encoder is kept; its output features are what the evaluation of Section 5.5 uses.

5.3.2 Masked Latent Prediction on Noisy Music

The standardized window is cut into $N = N_F N_T = 190$ patches. A random subset $\mathscr{M}$ of $|\mathscr{M}| = \text{round}(\rho N) = 133$ patches is masked with $\rho = 0.7$, and the remaining 57 patches form the visible set $\mathscr{V}$. The online encoder maps the visible patches to $z_{\mathscr{V}} = f_\theta(\tilde{x}_{\mathscr{V}})$, and the predictor, given mask tokens m and positional encodings p, predicts all token representations,

$$\hat{z} = g_\theta\big(\text{concat}(z_{\mathscr{V}}, m) + p\big), \tag{5.5}$$

of which only the masked ones, $\hat{z}_{\mathscr{M}}$, are used in the loss. The target encoder receives the masked patches only, $z_{\mathscr{M}} = f_\xi(\tilde{x}_{\mathscr{M}})$, and its output is standardized to zero mean and unit variance, giving $\tilde{z}_{\mathscr{M}}$. The loss is the mean squared error between the ℓ_2-normalized prediction and target,

$$\mathscr{L}_{\text{m2d}} = \frac{1}{|\mathscr{M}|}\sum_{j\in\mathscr{M}}\left\|\frac{\hat{z}_j}{\|\hat{z}_j\|_2} - \frac{\tilde{z}_j}{\|\tilde{z}_j\|_2}\right\|_2^2 = \frac{1}{|\mathscr{M}|}\sum_{j\in\mathscr{M}}\left(2 - 2\,\frac{\langle \hat{z}_j, \tilde{z}_j\rangle}{\|\hat{z}_j\|_2\,\|\tilde{z}_j\|_2}\right). \tag{5.6}$$

No gradient flows into the target encoder. Its parameters follow the online encoder by an exponential moving average,

$$\xi \leftarrow \tau_k \xi + (1-\tau_k)\,\theta, \qquad \tau_k = \tau_0 + (\tau_1 - \tau_0)\,\frac{k}{K}, \tag{5.7}$$

where k is the optimizer step, K the total number of steps, $\tau_0 = 0.99995$ and $\tau_1 = 0.99999$.

5.3.3 Background Noise and Denoising

To build the noisy input, a window of the noise corpus is mixed with the music window in the power domain, after the mean log level of each corpus has been removed so that the two sources start at comparable levels. With the music log-mel window $x^{(m)}$, the noise window $x^{(b)}$, their corpus means μ_m and μ_b, and the mixing ratio η,

$$x_{\text{mix}} = \log\Big((1-\eta)\exp\big(x^{(m)} - \mu_m\big) + \eta\exp\big(x^{(b)} - \mu_b\big)\Big), \qquad \tilde{x} = \frac{x_{\text{mix}}}{\sigma_{\text{AS}}}, \tag{5.8}$$

which is the mixing rule of M2D-X [4] followed by the same scale normalization as in Equation 5.2. Here μ_m is the AudioSet mean μ_{AS} of Table 5.2 and μ_b is the mean of the noise corpus. The noise windows are drawn without replacement by cycling through a shuffled list of the corpus, and short noise clips are repeated to fill the window. The ratio is fixed at $\eta = 0.3$ in the arms that use noise, the value of the respiratory-sound recipe [4]. Without a second view, the noise would only be an augmentation; the denoising character arises because the offline losses below are computed against the clean signal, so the encoder is asked to produce features that describe the music underneath the noise.

5.3.4 Frozen-Teacher Regularization

Both offline losses act on frame-level features. Let $\hat{Z}$ be the grid of $N_F \times N_T$ token representations in which the visible positions hold the encoder outputs $z_{\mathscr{V}}$ and the masked positions hold the predictions $\hat{z}_{\mathscr{M}}$. The feature of time step τ is the concatenation over the N_F frequency positions,

$$\hat{h}_\tau = \text{concat}\big(\hat{Z}_{1,\tau}, \ldots, \hat{Z}_{N_F,\tau}\big) \in \mathbb{R}^{N_F D}, \qquad N_F D = 5 \times 768 = 3840, \tag{5.9}$$

which is the same layout as the 3,840-dimensional frame features used in the linear evaluation of M2D [17]. The frozen original encoder f_{orig} processes the clean window with all patches visible and gives the target features $y_\tau^{\text{off}} \in \mathbb{R}^{3840}$. The regularization loss compares them after a linear map,

$$\mathscr{L}_{\text{off}} = \frac{1}{N_T}\sum_{\tau=1}^{N_T}\left\| \frac{W_{\text{off}}\hat{h}_\tau + b_{\text{off}}}{\|W_{\text{off}}\hat{h}_\tau + b_{\text{off}}\|_2} - \frac{y_\tau^{\text{off}}}{\|y_\tau^{\text{off}}\|_2}\right\|_2^2. \tag{5.10}$$

This term anchors the features of the adapted model to those of the original model on clean inputs, in the manner of learning without forgetting [47], while the first term moves them towards the music distribution. In the speech variant the same mechanism distils a domain model; here the teacher is the model itself, so the term is a regularizer, as in the respiratory-sound experiments [4].

5.3.5 Musical Teacher

The musical teacher adds a second offline task on the same frame features. A linear mapper predicts the pooled CQT target of Equation 5.4,

$$\mathscr{L}_{\text{cqt}} = \frac{1}{N_T K_c} \sum_{\tau=1}^{N_T} \left\| W_{\text{cqt}} \hat{h}_\tau + b_{\text{cqt}} - \bar{c}_\tau \right\|_2^2, \qquad K_c = 84. \tag{5.11}$$

Because $\hat{h}_\tau$ is built from encoder outputs at visible positions and from predictor outputs at masked positions, the term has two roles at once: it asks the encoder to recover the clean pitch content from noisy visible patches, and it asks the predictor to infer the pitch content of hidden regions. Since the target is standardized, the initial value of the loss is of the order of one, comparable to the other two terms, and the weight $\lambda_{\text{cqt}} = 1$ follows MERT's finding that a unit weight worked best among 1, 2 and 5 [7].

5.3.6 Joint Objective and Optimization

The full objective is the weighted sum

$$\mathscr{L} = \lambda_{\text{m2d}} \mathscr{L}_{\text{m2d}} + \lambda_{\text{off}} \mathscr{L}_{\text{off}} + \lambda_{\text{cqt}} \mathscr{L}_{\text{cqt}}. \tag{5.12}$$

The experimental arms of Section 5.4 differ only in η, λ_{off} and λ_{cqt}; all other settings are identical and are listed in Table 5.3. They follow the official M2D-X recipe [17, 4] with AdamW [48]. The learning rate is scaled with the effective batch size B_{eff} by the rule of the code base,

$$\text{lr} = \text{blr} \cdot \frac{B_{\text{eff}}}{256} = 3 \times 10^{-4} \cdot \frac{128}{256} = 1.5 \times 10^{-4}. \tag{5.13}$$

The effective batch size is kept at 128 even though the micro-batch is smaller, because the EMA update is applied once per optimizer step, so the number of steps affects the target network; the guide of the authors stresses that the effective batch size matters [17]. Algorithm 2 gives the procedure.

Table 5.3: Fixed settings of the further pre-training runs. Settings in the upper part follow the released M2D model and the M2D-X recipe; the lower part are the choices made for this study.

Item	Setting
Initial weights	M2D/0.7 (`m2d_vit_base-80x608p16x16-221006-mr7`): online encoder, predictor and target encoder
Network	ViT-Base encoder, 768-dimensional tokens, about 86 M parameters; MAE-style predictor
Input and masking	80×608 log-mel window, 16×16 patches ($N = 190$), masking ratio 0.7 (57 visible, 133 masked)
Optimizer	AdamW with $\beta = (0.9, 0.95)$, weight decay 0.05, gradient-norm clipping at 3.0
Learning rate	Base 3×10^{-4} scaled to 1.5×10^{-4}; 24 warm-up epochs; cosine decay to zero
EMA decay	Linear from 0.99995 to 0.99999 over training
Epochs and data per epoch	600 epochs of 10,000 virtual samples, that is, 6.0 M sample views and 46,875 optimizer steps
Batch	Micro-batch 32, gradient accumulation 4, effective batch 128
Noise	FSD50K development clips, $\eta = 0.3$ in arms with noise
Loss weights	$\lambda_{\text{m2d}} = 1$; $\lambda_{\text{off}} \in \{0, 1\}$; $\lambda_{\text{cqt}} \in \{0, 1\}$
Precision and hardware	Mixed precision on one RTX A4000 (16 GB)
Checkpoints	Saved every 100 epochs; epoch 600 is the reported checkpoint

Data: initial weights (θ_0, ξ_0); frozen teacher f_{orig}; music list $\mathscr{D}$ with log-mel and CQT arrays; noise list $\mathscr{B}$; η, λ_{m2d}, λ_{off}, λ_{cqt}; epochs E; epoch size n; micro-batch b; accumulation a
Result: further pre-trained encoder f_θ
$\theta \leftarrow \theta_0$; $\xi \leftarrow \xi_0$; initialize W_{off}, W_{cqt};
repeat $\mathscr{D}$ until the epoch list $\mathscr{D}_n$ has n entries;
for $e = 1$ **to** E **do**
  shuffle $\mathscr{D}_n$;
  foreach *micro-batch of b clips* **do**
    foreach *clip i in the micro-batch* **do**
      draw one start index; crop 608 frames of the log-mel array and of the CQT array at that index;
      take the next crop from the shuffled noise list $\mathscr{B}$ and form $\tilde{x}_i$ with Equation 5.8;
    end
    draw random masks; compute $\mathscr{L}_{\text{m2d}}$ with Equations 5.5 and 5.6;
    build $\hat{h}$ (Equation 5.9); compute y^{off} from the clean window with f_{orig} without gradient;
    compute $\mathscr{L}_{\text{off}}$ and $\mathscr{L}_{\text{cqt}}$ (Equations 5.10 and 5.11); form $\mathscr{L}$ (Equation 5.12);
    backpropagate $\mathscr{L}/a$ into θ, W_{off}, W_{cqt};
    if *a micro-batches have been accumulated* **then**
      clip the gradient norm; apply an AdamW step with the scheduled learning rate;
      update ξ with Equation 5.7;
    end
  end
  if *e is a multiple of 100* **then** save a checkpoint of f_θ;
end
return f_θ;

Algorithm 2: Further pre-training with the M2D-X-Music objective

5.4 Experimental Design

The experiments consist of a small set of further pre-training arms, defined in Table 5.4, that switch the components of the framework on and off one at a time, and of runs that vary the number of FMA tracks. All runs use the same track list and seeds and are compute-matched, that is, each processes the same 6.0 M sample views. The planned contrasts are A1 against A0 (naive continuation), A2 against A1 (background noise), A3 against A2 (frozen-teacher regularizer), A4 against A3 and A5 against A2 (musical teacher with and without the regularizer), and S1 to S3 against the best arm (number of tracks). The best arm, A^*, is the better of A3 and A4 by the mean music score on the validation splits only, so that the test splits take no part in any choice.

Table 5.4: Pre-training arms. Dashes mark runs without FMA pre-training. All arms use the same track list, the same random seed for data order and masks, and 6.0 M sample views.

Arm	η	λ_{off}	λ_{cqt}	Tracks	Purpose
A0	–	–	–	–	Unmodified M2D/0.7 (reference point)
A1	0	0	0	10,000	Naive continuation with the M2D loss only
A2	0.3	0	0	10,000	Adds background noise
A3	0.3	1	0	10,000	M2D-X: noise and frozen-teacher regularizer
A4	0.3	1	1	10,000	M2D-X-Music: adds the CQT musical teacher
A5	0.3	0	1	10,000	Musical teacher without the regularizer
S1–S3	0.3	as A^*	as A^*	1,000; 2,500; 5,000	Data-size study with the best arm A^*

The reported checkpoint of every arm is the one at epoch 600. If time is short, the arms are run in the order A3, A1, A4, A2, A5, then S1 to S3. Optional runs follow only if time remains: a noise pool without music-related FSD50K classes, a lower noise ratio $\eta = 0.1$, the FMA normalization statistics, a 20,000-track run, a second pre-training seed for A3 and A4, and the distillation of frozen MERT features as a second teacher. The checkpoints are compared with public music-specific models, namely MERT-v1-95M and MERT-v0-public [7, 11], MusicFM trained on FMA [8] and the music and general versions of MATPAC++ [12], which are evaluated with the same protocol and are never further trained.

5.5 Evaluation Protocol

Every model is evaluated with frozen features and a linear classifier, the protocol of the M2D authors [4], on the tasks of Table 5.5. The music tasks cover genre, instrument family, pitch and tagging; pitch is included because the CQT teacher targets exactly this information. The tasks are run with the EVAR toolkit [20], to which metadata for FMA-small and MagnaTagATune are added. Clips are cut into consecutive 6.08-s windows, each window is encoded with all patches visible, and the clip embedding is the temporal mean of the 3,840-dimensional frame features. A single linear layer is trained on the

standardized embeddings with Adam at a learning rate of 3×10^{-5} for at most 200 epochs, with early stopping after 20 epochs without improvement on the validation split, using the softmax cross-entropy loss for single-label tasks and the sigmoid binary cross-entropy loss for tagging. Each probe is trained with six random seeds.

Table 5.5: Downstream tasks. Splits are given as training, validation and test sizes where a fixed split exists. The GTZAN and NSynth figures are those used in the M2D evaluation [4]; the FMA-small split follows the dataset paper [13].

Task	Role	Labels	Metric	Split
GTZAN [42]	Music, genre	10 genres	Accuracy	Fault-filtered: 443 / 197 / 290
NSynth, instrument [45]	Music, timbre	11 instrument families	Accuracy	289,205 / 12,678 / 4,096
NSynth, pitch [45]	Music, pitch	MIDI pitch	Accuracy	Same notes as above
FMA-small genre [13]	Music, in-domain	8 genres	Accuracy	Official, artist-filtered: 6,400 / 800 / 800
MagnaTagATune [44]	Music, tagging	50 tags	mAP	One fixed published split

Accuracy is reported for single-label tasks and mean average precision (mAP) for tagging, in percent, as the mean over the six probe seeds with a 95 % confidence interval. With $s_u(m)$ the score of model m on task u, the music score is the mean over the music tasks $\mathcal{T}_{\text{mus}}$,

$$S_{\text{mus}}(m) = \frac{1}{|\mathcal{T}_{\text{mus}}|} \sum_{u \in \mathcal{T}_{\text{mus}}} s_u(m), \tag{5.14}$$

and, with m_0 the original M2D model, the gap-closure ratio

$$\text{GCR}_u(m; r) = \frac{s_u(m) - s_u(m_0)}{s_u(r) - s_u(m_0)} \tag{5.15}$$

expresses how far a further pre-trained model m moves from the original model towards a reference model r, from 0 (no change) to 1 (parity), and is computed only on tasks where the reference leads the original model by at least one point. A difference between two models is tested with a paired bootstrap over the test clips with 10,000 resamples [49] and is called supported only if its 95 % interval excludes zero. FMA-small is reported separately from the other music tasks because it shares its source with the pre-training data.

5.6 Conclusion

This chapter presented the proposed approach. A seeded, artist-capped procedure selects nested subsets of up to 10,000 tracks from the training split of the FMA medium subset, and a resumable pipeline converts them with the front-end of the original M2D model into log-mel arrays and CQT targets. The

M2D-X-Music framework further pre-trains M2D on music mixed with background noise, regularizes it with a frozen copy of the original model, and adds a linear head that regresses the CQT of the clean audio in the manner of the musical teacher of MERT. Compute-matched arms switch these components on and off and vary the number of tracks, and every checkpoint and reference model is evaluated with the same frozen-feature linear protocol on music tasks. The next chapter presents the high-level and low-level design of the system that implements this approach.

Chapter 6 High-Level and Low-Level Design

This chapter presents the design of the whole system: the research platform that carries out the method of Chapter 5 and the demonstration dashboard specified in Chapter 4. It first gives an overview and the considerations that shaped the design, then describes the architecture at system and subsystem level with two interaction scenarios. It then explains the main architectural strategies, the class diagram, and the policies and tactics that govern the implementation. All software is written in Python, and all models are trained and run with PyTorch [19]. Wherever possible, the design extends the public M2D code base [17] and the EVAR evaluation toolkit [20] rather than replacing them, and it borrows the conventions of the MERT [23] and MATPAC++ [50] code bases where they fit.

6.1 System Overview

The system consists of four subsystems. The data preparation pipeline turns the Free Music Archive and the noise corpus into the input format of M2D. The further pre-training framework trains one checkpoint per further pre-training run. The evaluation and analysis subsystem measures every checkpoint and every reference model with the same protocol. The demonstration dashboard presents the results and runs the four demonstration tasks. The subsystems communicate only through files on disk: spectrogram arrays, CSV lists of training files, model checkpoints, score files and saved linear probes. Each subsystem can therefore be run, tested and replaced on its own, and every result can be traced to the files that produced it.

6.1.1 Data Preparation Pipeline

A command-line package, `fmaprep`, runs the steps of Section 5.2 as named stages: select draws the nested 1,000- to 10,000-track subsets, audit decodes and screens each track, convert writes the log-mel and CQT arrays, and check runs the integrity checks. Each stage reads the outputs of the earlier stages and writes to its own folder, so an interrupted run resumes where it stopped.

6.1.2 Further Pre-training Framework

The framework is the M2D code base with a small extension that adds the musical teacher, the CQT targets and fixed normalization statistics. Each training run produces the checkpoints of one configuration of Table 5.4, and the runs differ only in the command-line options of the training script.

6.1.3 Evaluation and Analysis

The EVAR toolkit computes frozen embeddings, trains the linear probes and records the scores. Small wrapper classes make the reference models available through the same interface, and analysis scripts compute the aggregates, confidence intervals and gap-closure ratios of Section 5.5.

6.1.4 Demonstration Dashboard

A FastAPI backend serves a single-page HTML and JavaScript frontend, reads the imported results from a SQLite database and runs the encoders and probes of the evaluation for the four demonstration tasks (Chapter 4).

6.2 Design Considerations

The design must fit a single GPU and a three-month schedule, keep the comparison between runs fair, and produce results that can be reproduced and traced. The considerations below shaped it.

6.2.1 Assumptions and Dependencies

The design relies on the following assumptions and dependencies.

- The M2D code base at a fixed commit, together with the MAE source files that it patches [1], the timm library and nnAudio [21]; the EVAR toolkit; PyTorch with CUDA; and the Hugging Face `transformers` library [51] for loading the MERT and MATPAC++ checkpoints. A change in the training interface of the M2D code would affect the extension built on it, which is why the commit is fixed.
- Linux on the workstation, with the NVIDIA driver and CUDA runtime.
- One NVIDIA RTX A4000 with 16 GB of memory for training, evaluation and the dashboard; the dashboard can also run on a CPU, more slowly.
- The FMA medium subset and metadata, the FSD50K development set and the downstream datasets of Table 5.5 are available locally at configured paths, together with the released checkpoints of M2D/0.7 and of the reference models.
- Researchers use the command line and configuration files; visitors use only the web interface of the dashboard.
- The further pre-trained model shown by the dashboard is chosen only at the end of the experiments; optional runs (Section 5.4), the distillation of MERT features as a second teacher, and

further dashboard tasks may be added later. The design keeps each of these to a new script, a new option or new database rows.

6.2.2 General Constraints

The design is subject to the following constraints.

- The online, target and predictor networks, the frozen teacher and the two linear heads must fit into 16 GB during training. This fixes the micro-batch at 32 with four accumulation steps (Table 5.3), mixed precision, and a teacher that runs without gradients on the same GPU.
- Spectrograms must be computed with exactly the front-end of the original M2D pre-training (Table 5.2); any other implementation would add a distribution shift that cannot be separated from the effect under study.
- About 10 GB of half-precision arrays for 10,000 tracks (Section 5.2), plus the checkpoints of every run, which hold the online and target networks and the optimizer state and are saved every 100 epochs.
- FMA audio is used under its Creative Commons licenses, and the MERT weights are licensed for non-commercial use [11]; no audio is redistributed except the openly licensed dashboard samples.
- The MATPAC++ inference code pins an old version of timm that conflicts with the version needed by the M2D code [50], so the two cannot share one Python environment.
- All runs must use the same track list and seeds, every stage must verify its outputs, and the dashboard must meet the non-functional requirements of Section 4.4.

6.2.3 Goals and Guidelines

The design follows the goals and guidelines below.

- The official recipe is only reproduced faithfully if its code is reused, so new code is added as subclasses and options of the existing code rather than as a re-implementation.
- Runs differ only in command-line options, never in code paths that are switched by editing files, so that a comparison between runs is a comparison of exactly the intended factor.
- Training, evaluation and the dashboard compute spectrograms and embeddings with the same functions, so that the dashboard demonstrates the models that were evaluated.
- Plain files, command-line scripts and a file-based database are used instead of services, because the data are small and must be easy to inspect.

- Missing files, shape mismatches, non-finite losses and unknown codes stop the run with a clear message instead of being skipped silently.
- Every run stores its options, the code version and a hash of its track list next to its outputs.

6.2.4 Development Methods

The project follows an iterative, research-driven process. The data pipeline and its tests are built first; a pilot run of a few epochs then measures time and memory on the A4000 (Section 5.3); the runs are carried out in the order fixed in Chapter 5, with evaluation of each checkpoint as it appears; and the dashboard is built in small steps once the first probes exist, starting with the original M2D model (Section 4.10). A strict waterfall process was rejected because the pilot run can change the schedule, for example by reducing the number of epochs for all runs. Parts whose correctness can be checked automatically, such as subset selection, shapes and losses, are developed with unit tests; the rest is checked with spectrogram galleries, training curves and the label-free feature statistics logged by the training script.

6.3 System Architecture

Figure 6.1 shows the layered architecture. The data layer holds the raw corpora, the downstream datasets and the released checkpoints. The preparation layer, the `fmaprep` pipeline, turns the corpora into arrays and file lists. The learning layer carries out each further pre-training run with the frozen original model as teacher. The evaluation layer runs EVAR on the checkpoints and the reference models, and its analysis scripts compute the statistics. The presentation layer imports the results into the dashboard. Each layer depends only on the files written by the layers below it.

This decomposition was chosen because the three expensive steps, conversion, training and embedding extraction, have very different costs and must not be repeated when a later step changes. With files between the layers, a new run reuses the converted data, and a new statistic reuses the cached embeddings. A single training script that reads MP3 files and evaluates at the end was rejected: it would mix data handling with model code, repeat the conversion for every run, and make it hard to show that all runs saw the same data.

6.3.1 Subsystem Architecture

Figure 6.2 shows the modules of the training subsystem; the data flow of the objective itself is given in Figure 5.3. The training script `train_audio.py` of the M2D code base parses the options of a run, creates the run folder and builds three objects. The dataset `MusicMixedSpecDataset` extends

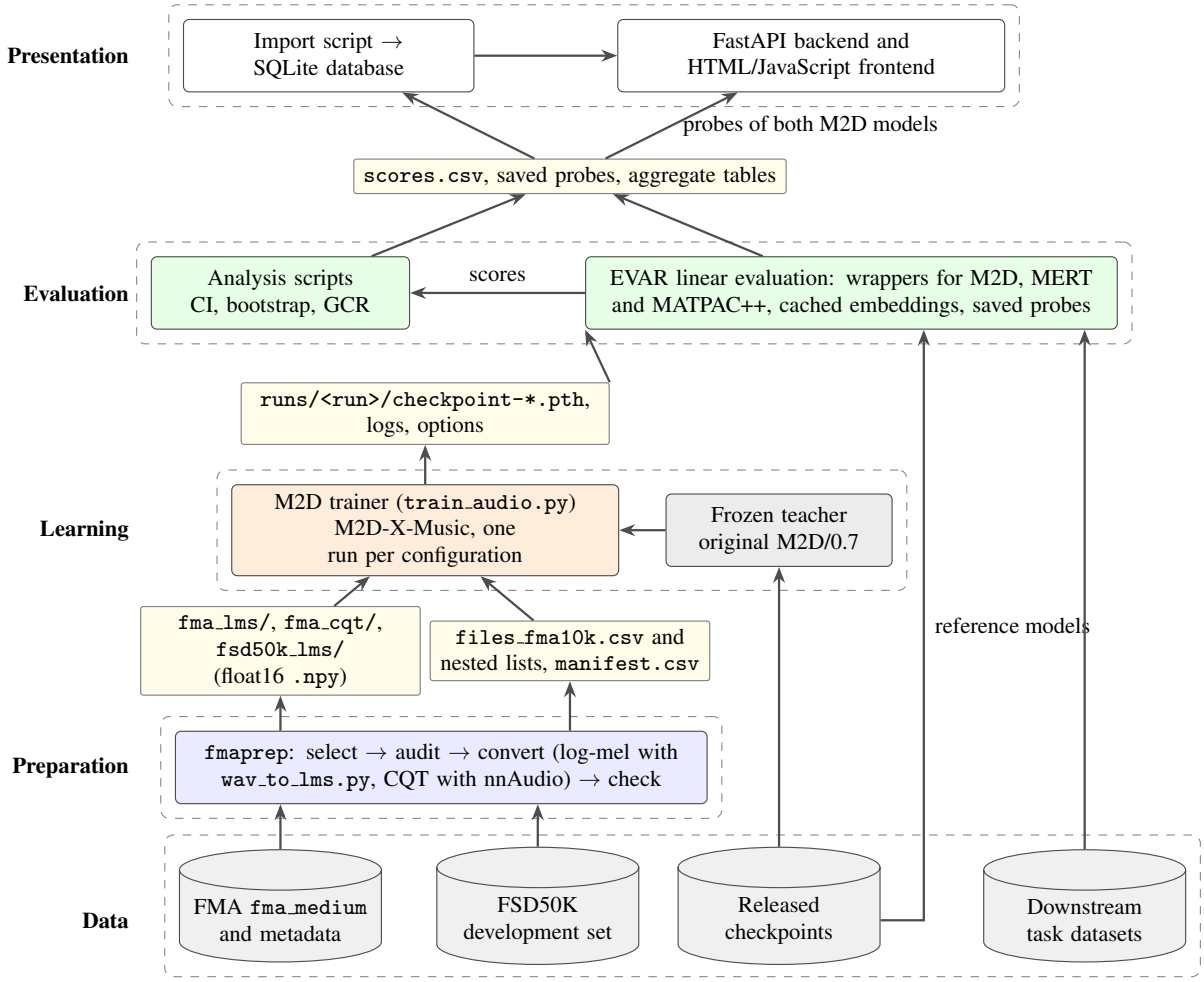


Figure 6.1: Layered system architecture, read from bottom to top. Each layer reads only the files written by the layers below it; yellow boxes are files.

the mixed-noise dataset of M2D-X: it reads the music and noise spectrograms through the existing `SpectrogramDataset`, mixes them with the existing log-domain mixing function, crops the CQT array at the same start frame, and applies the fixed AudioSet statistics. The model `M2D_X_Music_ViT` extends `M2D_X_ViT`, which already contains the online encoder, the predictor, the EMA target encoder and the offline linear head, with a second linear head for the CQT target. The teacher is the existing `RuntimeM2D` class loaded with the original M2D/0.7 weights and frozen. A training loop adapted from `train_one_epoch_m2dx` passes the three losses to the optimizer, updates the target encoder and logs each loss separately. Checkpoints are written every 100 epochs by the existing `misc.save_model`, which also starts the evaluation hook `quick_eval.sh`.

The stages of `fmaprep` follow Figure 5.2. Each stage is a class with a `run` method and writes a completion marker, so finished stages are skipped unless forced. All paths, seeds and thresholds are read from one YAML file. The convert stage calls the unchanged converter of `wav_to_lms.py` for log-mel spectrograms and an nnAudio CQT layer with the settings of the MERT musical teacher [23], and writes

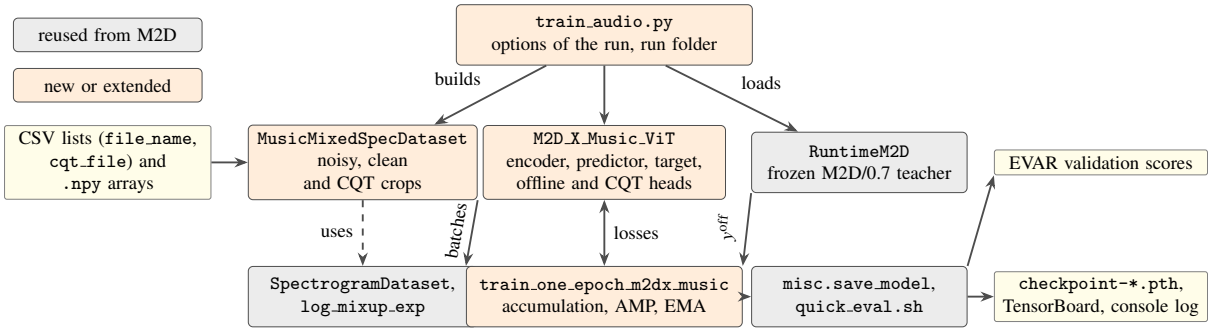


Figure 6.2: Modules of the further pre-training subsystem. Grey boxes are reused from the M2D code base, orange boxes are new or extended, and yellow boxes are files.

the training list in the CSV format of the M2D code, extended with a column for the CQT file.
Each model is described by an EVAR configuration file that names its wrapper class, weight file, sampling rate and normalization statistics. The M2D checkpoints use the existing wrapper `AR_M2D`; new wrappers `AR_MERT` and `AR_MATPAC` load the reference models from the Hugging Face hub and return the features of a chosen layer. Task metadata for FMA-small and MagnaTagATune are added as CSV files in the format of the existing tasks. EVAR caches the embeddings of each model and task, trains the probes with the protocol of Section 5.5, saves the trained linear layers on request, and appends every score to one results file, which the analysis scripts read.

The backend has four modules: an API module with the routes for results, tasks, samples and predictions; an inference module whose `InferenceService` holds the loaded encoders and probes and processes one request at a time; a repository module that maps the tables of Figure 4.8 with SQLAlchemy; and the import script of use case UC-06. The frontend consists of one HTML page, one script and one style sheet, and it obtains all data through the JSON API.

6.3.2 Sequence Diagrams

Figures 6.3 and 6.4 show the two central interactions: one step of further pre-training, and one prediction request of the dashboard.

Further pre-training (Figure 6.3) involves five participants: the training loop, the music dataset, the frozen teacher, the model, and the optimizer with the gradient scaler. In a loop over epochs and an inner loop over micro-batches, the training loop takes a batch of noisy, clean and CQT crops; the teacher encodes the clean crop without gradient; and the model masks, encodes and predicts on the noisy crop and returns the total loss and its three terms. The scaled loss is backpropagated, and every fourth micro-batch the gradients are clipped, the AdamW step is taken and the target encoder is updated. Every 100

epochs a checkpoint is saved and a quick evaluation is started.

A prediction request (Figure 6.4) involves six participants: the visitor, the browser, the FastAPI route, the inference service, the encoder with its probe, and the SQLite database. The browser posts the clip, task and models, and the route checks and decodes the audio. If the file is invalid, an error message is returned and shown. Otherwise the job is queued, and for each selected model the encoder computes the clip embedding and the probe returns the scores. The request is logged, the uploaded file is deleted, and the browser shows the bars or the side-by-side comparison.

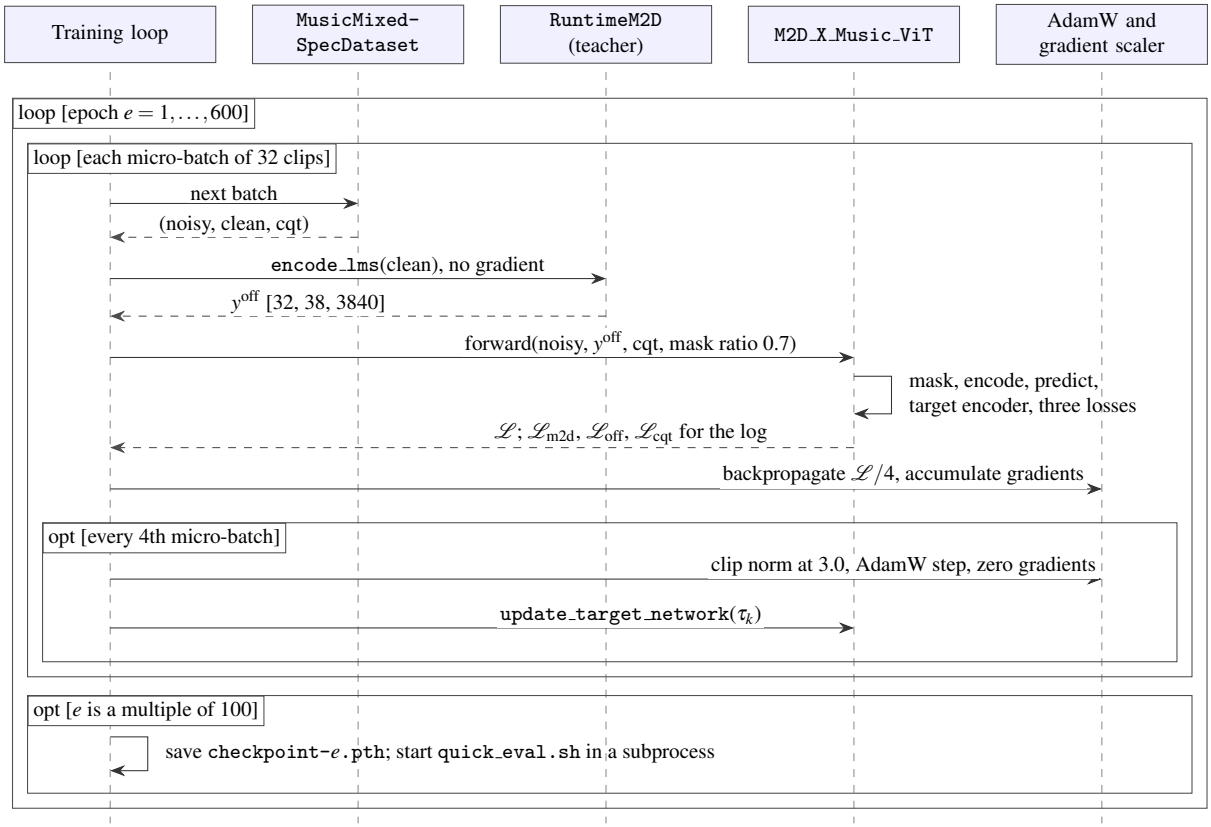


Figure 6.3: Sequence diagram of one further pre-training step. A batch passes through the frozen teacher and the model, and an optimizer step is taken every fourth micro-batch.

6.4 Architectural Strategies

The strategies below have the largest effect on the structure of the system. For each, the alternatives that were considered are given.

6.4.1 Extending the M2D Code Base

The further pre-training is implemented as a small extension of the M2D code base: a model subclass, a dataset subclass, an adapted training loop and three new options (the CQT loss weight, the CQT column

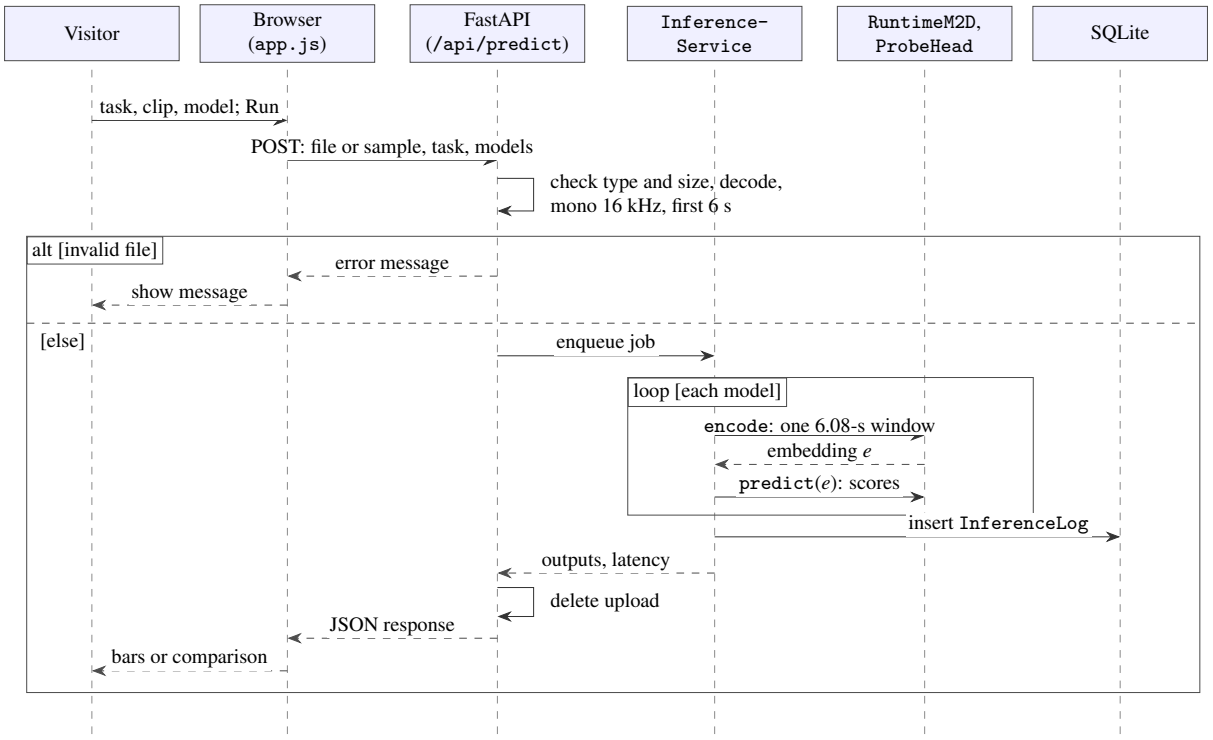


Figure 6.4: Sequence diagram of a dashboard prediction request (UC-04, UC-05). The clip is validated, embedded and scored by each selected model, and the result is logged and shown.

of the file list, and the use of the fixed normalization statistics given by the existing `--norm_stats` option). Everything else, from masking to the EMA schedule and checkpointing, is the code that produced the published M2D-X results [4], and the runs without the musical teacher use it unchanged except for the normalization statistics. The musical-teacher head copies the structure of the CQT predictor of MERT, a single linear layer from the features to 84 CQT bins [23]. Three alternatives were rejected. Porting the method to the fairseq framework [52], on which the MERT and MATPAC++ training code is built, would require converting the M2D weights and re-implementing the recipe. Writing a new training loop from scratch would risk small deviations from the recipe that could not be told apart from the effects under study. A framework such as PyTorch Lightning would add a layer of indirection around a loop that already exists and works.

6.4.2 Offline Precomputation of Inputs and Targets

Log-mel spectrograms and CQT targets are computed once and stored as half-precision arrays, and the data loader only crops, mixes and normalizes them. This follows the M2D code, which also trains from precomputed spectrograms, and it keeps the GPU free for the networks. The alternative of the MERT implementation, computing the CQT on the GPU at every step, was rejected: with 10,000 tracks the one-time conversion is cheap, whereas a per-step transform would cost memory and time on a 16-GB

GPU for every one of the 600 epochs. Mixing the noise at load time, rather than storing noisy copies, keeps the noise samples different in every epoch, as in the official recipe.

6.4.3 Experiments as Command Lines and Run Folders

Each further pre-training run is a short shell script that calls the training script with its options, for example the noise ratio, the loss weights, the file list and the seed; the configurations of Table 5.4 differ only in these options. The training script writes the options, the commit of the code and a hash of the track list into the run folder next to the checkpoints and logs, so that the folder is a complete record of the run. Configuration through Hydra YAML files, as in the fairseq-based code of MERT and MATPAC++, was considered but not adopted, because the M2D code is configured with command-line options and two configuration systems would have to be kept consistent.

6.4.4 One Evaluation Interface for All Models

All models, whether further pre-trained checkpoints or references, are evaluated through the model interface of EVAR, in which a wrapper class turns a batch of waveforms into frame features. The same probe code, splits and seeds therefore apply to every model, and the probes saved by EVAR are the ones loaded by the dashboard. The reference models are loaded from the Hugging Face hub with the `transformers` library [51] in a separate environment, because of the library conflict noted above; this environment writes only embeddings to the shared cache. Evaluating with the MARBLE toolkit, for which EVAR provides a plug-in, was rejected for this study because of its cost on one GPU and because scores of its revised versions are not comparable with each other (Chapter 3).

6.4.5 Files as Interfaces and Single-Worker Serving

The subsystems exchange only files, and the dashboard reads only the files exported by the evaluation and its own database. Training curves are kept in TensorBoard logs, which the M2D code already writes, rather than in an experiment-tracking server, which would add a service to maintain. In the dashboard, the encoders and probes are loaded once at start-up and all inference requests pass through one queue processed by a single worker, which bounds the GPU memory and satisfies the concurrency requirement of Section 4.4. A task queue with a separate broker, such as Celery with Redis, and loading the models for each request were both rejected: the first is unnecessary for a handful of visitors, and the second would make every prediction slow.

6.5 Domain Model/Class Diagram

Figure 6.5 shows the main classes. Grey classes are reused unchanged from the M2D code base and EVAR; white classes are new. In the data part, `MusicMixedSpecDataset` extends `MixedSpecDataset`, which is composed of two `SpectrogramDataset` objects for music and noise, and the preparation stages share the abstract class `Stage`. In the model part, `M2D_X_Music_ViT` extends `M2D_X_ViT` (itself derived from `M2DViT`) with the CQT head, and the training loop uses the dataset, the model and `RuntimeM2D` as teacher. In the evaluation part, the wrappers derive from `BaseAudioRepr`. In the dashboard part, `InferenceService` holds encoders and `ProbeHead` objects and writes through `ResultRepository`, which `ResultImporter` also uses. `RuntimeM2D` appears three times: as the teacher during training, inside `AR_M2D` during evaluation, and inside `InferenceService` in the dashboard. This single encoder path is what makes the dashboard demonstrate exactly the evaluated models.

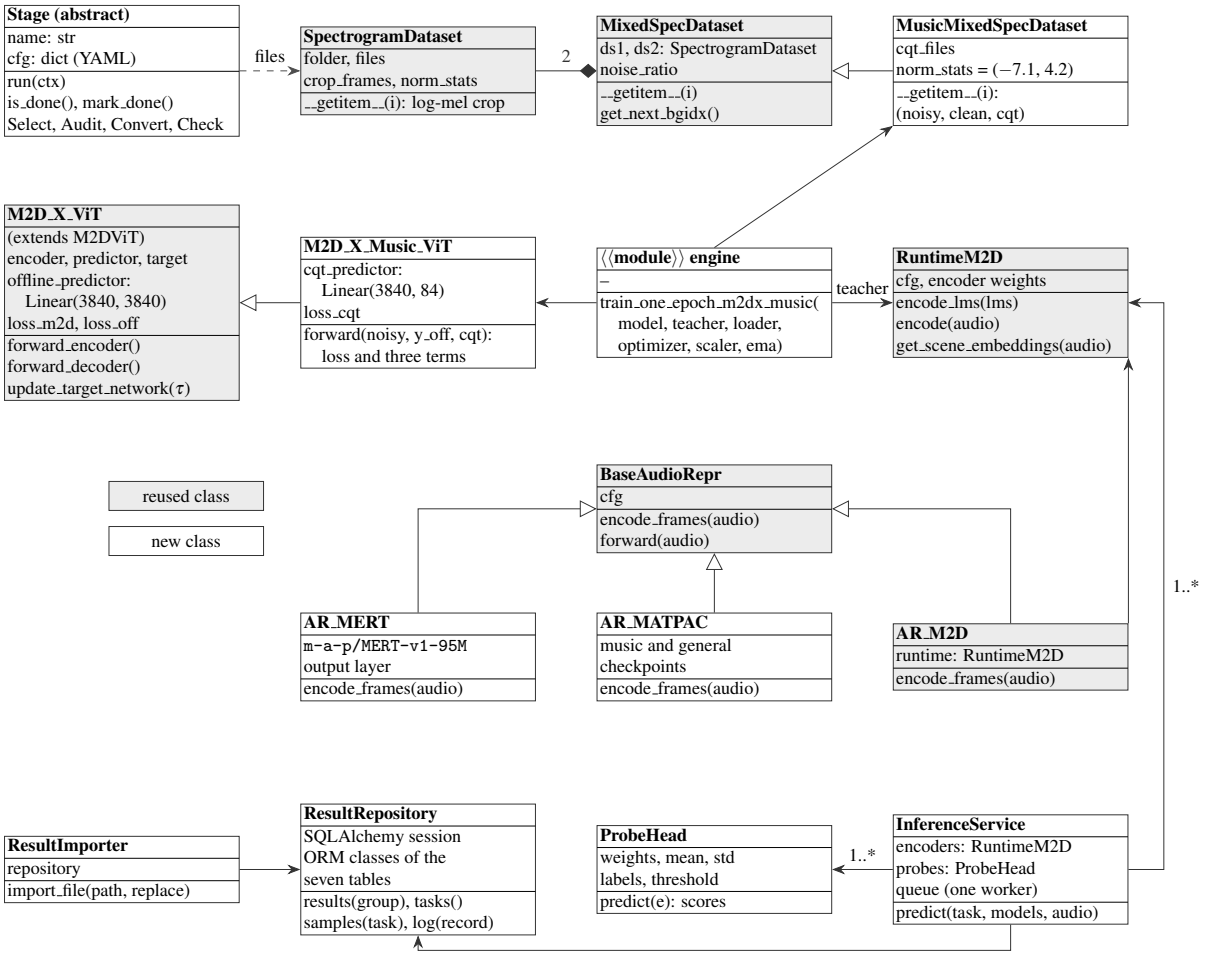


Figure 6.5: Class diagram of the system. Grey classes are reused and white classes are new; triangles mark inheritance, the diamond composition, solid arrows associations and the dashed arrow a file dependency.

6.6 Policies and Tactics

These policies and tactics affect the details of the implementation rather than its overall structure.

6.6.1 Choice of Tools

Python 3.10 or above and PyTorch [19] are used throughout, with the versions of PyTorch, timm and nnAudio pinned in a conda environment file. A second environment with the versions required by the reference models is used only for extracting their embeddings. TensorBoard records training curves, pytest runs the tests, and the dashboard uses FastAPI with Uvicorn, SQLAlchemy with SQLite, and Chart.js, as listed in Section 4.7. fairseq is not installed, because no fairseq-based model is trained; MERT and MATPAC++ are only used through their published checkpoints.

6.6.2 Coding Guidelines and Conventions

The code follows PEP 8 and is checked with a linter; functions carry type hints and docstrings. Extensions of the M2D code follow its naming, for example `M2D_X_Music_ViT` with the factory function `m2d_x_music_vit_base` and the option `--loss_cqt`, so that they read like the existing `M2D_X_ViT`, `m2d_x_vit_base` and `--loss_off`. Changes to upstream files are kept in a patch file, in the same way as the M2D code base distributes its own changes to the MAE files, and new classes are placed in new files. Paths are never hard-coded; they come from options or the configuration file. Every source of randomness takes an explicit seed.

6.6.3 Testing

Unit tests with pytest cover the parts whose correctness can be checked automatically: the subset selection (nested lists, artist cap and identical output for the same seed); the audit rules on synthetic silent, short and duplicate files; the equal frame counts of log-mel and CQT arrays; the mixing function against Equation 5.8; the shapes of the CQT head output ($32 \times 38 \times 84$); and checkpoint loading, which must accept the M2D/0.7 weights with only the new heads missing. Two tests guard the comparison between runs. The first checks that, with the CQT weight set to zero, the loss of the new model equals that of `M2D_X_ViT` on the same batch and masks, so that the extension cannot change the runs without the musical teacher. The second checks which view each network receives: the dataset returns its views by name rather than by position, and the test asserts that the online network gets the noisy view and the teacher the clean view, because swapping two tensors of the same shape would raise no error. A smoke test runs one epoch on a list of a few tracks, and the pilot run of Chapter 5 serves as the system test of training. For the dashboard, API tests use the FastAPI test client, and a consistency test checks that the

dashboard reproduces the cached EVAR embeddings of a few test clips. The full test cases are given in the next chapter.

6.6.4 Requirements Traceability

Each research question is linked to the further pre-training runs and comparisons that answer it (Section 5.4), each contrast to rows of the results file, and each row to its checkpoint, task, split and probe seed. The functional requirements FR-01 to FR-18 of the dashboard (Table 4.1) map to the modules of Section 6.3: results views to the API and repository, FR-06 to FR-15 to the API and inference modules, FR-16 to the repository, and FR-17 to the import script. The database keeps the source file of every imported score, so every number on the dashboard can be traced back to a run.

6.6.5 Source Organization and Build

The repository contains the following folders: `fmaprep/` (preparation stages and their YAML configuration); `m2d/` (a clone of the M2D code at a fixed commit, the downloaded MAE files and the patch `patch_music.diff`); `evar/` (a clone of EVAR with the new wrappers, configuration files and task metadata); `runs/` (one script per further pre-training run); `analysis/` (statistics and report tables); `dashboard/backend/` and `dashboard/frontend/`; and `tests/`. Data, arrays, checkpoints and caches are kept outside version control. The system is built by creating the conda environments, cloning the two upstream repositories, downloading the MAE files and applying the patches, as in the setup of the M2D code base [17]. Then `python -m fmaprep all` prepares the data, `bash runs/<run>.sh` carries out one further pre-training run, the EVAR command `lineareval.py` evaluates a checkpoint on a task, and `uvicorn` starts the dashboard.

6.6.6 Maintenance

A new further pre-training run is added as one script in `runs/`, a new reference model as one wrapper class and configuration file, and a new dashboard task as task metadata, a saved probe and database rows. If the upstream code changes, the patch is re-applied at the new commit and the unit tests show whether the extension still behaves as before.

6.7 Conclusion

This chapter presented the design of the system. A layered architecture separates data preparation, further pre-training, evaluation and presentation, which communicate only through files, so that each expensive step runs once and every result can be traced. The main strategies are a small extension of the

M2D code base instead of a new framework, offline precomputation of spectrograms and CQT targets, experiments defined by command-line options and recorded in run folders, one evaluation interface for all models, and single-worker serving in the dashboard. The sequence and class diagrams show how the components collaborate, with one encoder path shared by training, evaluation and the dashboard, and the policies and tactics fix the tools, conventions, tests and traceability. Together with the method of Chapter 5 and the requirements of Chapter 4, this design is the basis for the implementation and the test cases in the next chapter.

Bibliography

[1] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16000–16009, 2022.

[2] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli, "data2vec: A general framework for self-supervised learning in speech, vision and language," in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 162, pp. 1298–1312, 2022.

[3] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, "Masked Modeling Duo: Learning representations by encouraging both networks to model the input," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2023.

[4] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, "Masked Modeling Duo: Towards a universal audio pre-training framework," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2391–2406, 2024.

[5] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 776–780, 2017.

[6] J. Turian, J. Shier, H. R. Khan, B. Raj, B. W. Schuller, *et al.*, "HEAR: Holistic evaluation of audio representations," in *Proc. NeurIPS 2021 Competitions and Demonstrations Track, PMLR*, vol. 176, pp. 125–145, 2022.

[7] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, N. Gyenge, R. Dannenberg, R. Liu, W. Chen, G. Xia, Y. Shi, W. Huang, Z. Wang, Y. Guo, and J. Fu, "MERT: Acoustic music understanding model with large-scale self-supervised training," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[8] M. Won, Y.-N. Hung, and D. Le, "A foundation model for music informatics," in *Proc.*

IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2024. arXiv:2311.03318.

[9] H. Zhu, Y. Zhou, H. Chen, J. Yu, Z. Ma, R. Gu, Y. Luo, W. Tan, and X. Chen, "MuQ: Self-supervised music representation learning with Mel residual vector quantization." arXiv preprint arXiv:2501.01108, 2025.

[10] Y. Gu, J. Zhang, J. Li, Z. Wu, and L. Juvela, "Frequency-aware self-supervised music representation learning." arXiv preprint arXiv:2606.25713, 2026. Preprint; the model was first named PupuJEPA and renamed PupuM2D in version 3.

[11] m-a-p, "MERT-v0-public and MERT-v1-95m model cards." https://huggingface.co/m-a-p/MERT-v0-public. Accessed: October 2026.

[12] A. Quelennec, P. Chouteau, G. Peeters, and S. Essid, "MATPAC++: Enhanced masked latent prediction for self-supervised audio representation learning." arXiv preprint arXiv:2508.12709, 2025. Preprint, under review.

[13] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2017.

[14] Y. Zhang, Y. Gao, W. Zhou, Z. Yuan, K. Imoto, and T. Kawahara, "SONAR: Self-distilled continual pre-training for domain adaptive audio representation." arXiv preprint arXiv:2509.15703, 2025.

[15] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, "FSD50K: An open dataset of human-labeled sound events," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 829–852, 2022.

[16] R. Yuan, Y. Ma, Y. Li, G. Zhang, X. Chen, H. Yin, *et al.*, "MARBLE: Music audio representation benchmark for universal evaluation," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[17] NTT Communication Science Laboratories, "M2D: Masked modeling duo, official implementation and pre-trained weights." https://github.com/nttcslab/m2d. Accessed: October 2026.

[18] C. Kereliuk, B. L. Sturm, and J. Larsen, "Deep learning and music adversaries," *IEEE Transactions on Multimedia*, vol. 17, no. 11, pp. 2059–2071, 2015.

[19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance

deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.

[20] NTT Communication Science Laboratories, "EVAR: Evaluation of audio representations." https://github.com/nttcslab/eval-audio-repr. Accessed: October 2026.

[21] K. W. Cheuk, H. Anderson, K. Agres, and D. Herremans, "nnAudio: An on-the-fly GPU audio to spectrogram conversion toolbox using 1D convolutional neural networks," *IEEE Access*, vol. 8, pp. 161981–162003, 2020.

[22] United Nations General Assembly, "Transforming our world: The 2030 agenda for sustainable development." Resolution A/RES/70/1, 2015.

[23] Y. Li *et al.*, "MERT: official implementation (fairseq plug-in) and training scripts." https://github.com/yizhilll/MERT. Accessed: October 2026.

[24] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, "Masked autoencoders that listen," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[25] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, W. Che, X. Yu, and F. Wei, "BEATs: Audio pre-training with acoustic tokenizers," in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 202, pp. 5178–5193, 2023.

[26] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, "Masked spectrogram modeling using masked autoencoders for learning general-purpose audio representation," in *HEAR: Holistic Evaluation of Audio Representations (NeurIPS 2021 Competition), PMLR*, vol. 166, pp. 1–24, 2022.

[27] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, "Masked Modeling Duo for speech: Specializing general-purpose audio representation to speech using denoising distillation," in *Proc. Interspeech*, pp. 1294–1298, 2023.

[28] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, "Investigating design choices in joint-embedding predictive architectures for general audio representation learning," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing Workshops (ICASSPW)*, pp. 680–684, 2024.

[29] L. Tuncay, E. Labbé, E. Benetos, and T. Pellegrini, "Audio-JEPA: Joint-embedding predictive architecture for audio representation learning." arXiv preprint arXiv:2507.02915, 2025.

[30] X-ARES developers, "X-ARES: Extensive audio representation and evaluation suite." https://github.com/jimbozhang/xares. Accessed: October 2026.

[31] A. Quelennec, P. Chouteau, G. Peeters, and S. Essid, "Masked latent prediction and classification for self-supervised audio representation learning," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5, 2025.

[32] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9630–9640, 2021.

[33] D. Perera, V. Letzelter, T. Mariotte, A. Cortés, M. Chen, S. Essid, and G. Richard, "Annealed multiple choice learning: Overcoming limitations of winner-takes-all with annealing," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 10541–10574, 2024.

[34] R. Castellon, C. Donahue, and P. Liang, "Codified audio language modeling learns useful representations for music information retrieval," in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 88–96, 2021.

[35] M. C. McCallum, F. Korzeniowski, S. Oramas, F. Gouyon, and A. F. Ehmann, "Supervised and unsupervised learning of audio representations for music understanding," in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 256–263, 2022.

[36] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *Transactions on Machine Learning Research*, 2023.

[37] C.-C. Chiu, J. Qin, Y. Zhang, J. Yu, and Y. Wu, "Self-supervised learning with random-projection quantizer for speech recognition," in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 162, pp. 3915–3924, 2022.

[38] W.-N. Hsu, A. Sriram, A. Baevski, T. Likhomanenko, Q. Xu, V. Pratap, J. Kahn, A. Lee, R. Collobert, G. Synnaeve, and M. Auli, "Robust wav2vec 2.0: Analyzing domain shift in self-supervised pre-training," in *Proc. Interspeech*, pp. 721–725, 2021.

[39] S. Lee, M. Kang, J. Lee, S. J. Hwang, and K. Kawaguchi, "Self-distillation for further pre-training of transformers," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[40] M. Defferrard *et al.*, "FMA: A dataset for music analysis, repository and download page." https://github.com/mdeff/fma. Accessed: October 2026.

[41] I. A. P. Santana *et al.*, "Music4All: A new music database and its applications," in *Proc. International Conference on Systems, Signals and Image Processing (IWSSIP)*, 2020.

[42] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.

[43] B. L. Sturm, "The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use." arXiv preprint arXiv:1306.1461, 2013.

[44] E. Law, K. West, M. Mandel, M. Bay, and J. S. Downie, "Evaluation of algorithms using games: The case of music tagging," in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, pp. 387–392, 2009.

[45] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, "Neural audio synthesis of musical notes with WaveNet autoencoders," in *Proc. International Conference on Machine Learning (ICML), PMLR*, vol. 70, pp. 1068–1077, 2017.

[46] J. C. Brown, "Calculation of a constant Q spectral transform," *The Journal of the Acoustical Society of America*, vol. 89, no. 1, pp. 425–434, 1991.

[47] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.

[48] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[49] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

[50] A. Quelennec, "MATPAC: Masked latent prediction and classification, code and checkpoints." https://github.com/aurianworld/matpac. Accessed: October 2026.

[51] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pp. 38–45, 2020.

[52] M. Ott, S. Edunov, A. Baevski, A. Fan, S. Gross, N. Ng, D. Grangier, and M. Auli, "fairseq: A fast, extensible toolkit for sequence modeling," in *Proc. Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT): Demonstrations*, pp. 48–53, 2019.